

4.1 WHAT IS DATAWEAVE?



DataWeave is MuleSoft's powerful data transformation language. It enables you to **transform**, **map** and **format** data between different formats and structures with ease.

INTRODUCTION

- ✓ DataWeave is the data transformation language of MuleSoft.
- ✓ It is used to manipulate and transform data in a readable, concise and declarative way.
- ✓ Introduced in Mule 4 and used across the Mule runtime.
- ✓ Replaces the legacy MEL (Mule Expression Language) for transformations.

WHY DATAWEAVE?

- ✓ Easy to learn and use
- ✓ Less code, more output
- ✓ Readability & maintainability
- ✓ Powerful built-in functions
- ✓ Supports multiple data formats
- ✓ High performance & streaming
- ✓ Strong typing & error handling

SIMPLE EXAMPLE

Input (JSON)

```
{
  "id": 101,
  "name": "John Doe",
  "email": "john@demo.com",
  "skills": ["MuleSoft", "Java"]
}
```

Output (JSON)

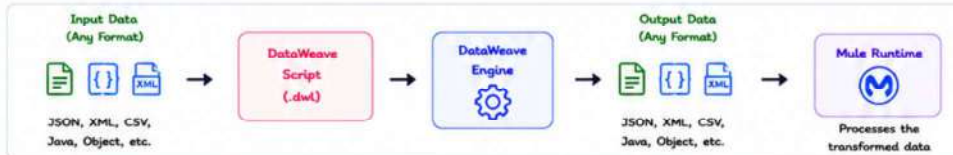
```
{
  "id": 101,
  "fullName": "John Doe",
  "contact": "john@demo.com",
  "skillCount": 2
}
```

DataWeave Script

```
%dw 2.0
output json
---
id: payload.id,
fullName: payload.name,
contact: payload.email,
skillCount: sizeOf(payload.skills)
```

💡 DataWeave turns complex transformations into simple and expressive scripts.

DATAWEAVE ARCHITECTURE



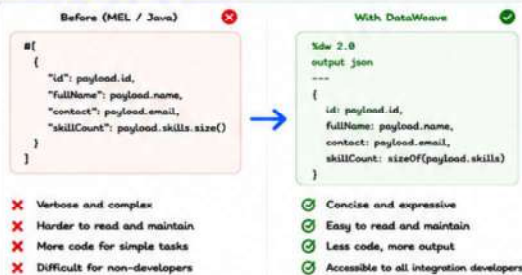
DATAWEAVE OF DATAWEAVE

- Declarative**
Focus on what to transform, not how to do it.
- Powerful Functions**
Rich set of built-in functions for strings, arrays, objects, dates, and more.
- Multiple Data Formats**
Supports JSON, XML, CSV, Java, and more. Easily convert between formats.
- High Performance**
Optimized execution engine with streaming support for large payloads.
- Easy to Read**
Human-friendly syntax. Less code, more clarity.
- Strong Typing**
Type-safe scripts with early error detection.
- Extensible**
Create custom functions and reuse across projects.
- Maintainable**
Clean, modular and easy to maintain transformations.

USE CASES

- ✓ Transform API requests and responses
- ✓ Map between different data models
- ✓ Data enrichment and aggregation
- ✓ Format data for reports and documents
- ✓ Integrate with multiple systems
- ✓ Convert between different data formats
- ✓ Handle complex business transformations

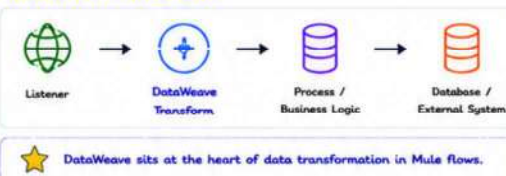
BEFORE & AFTER COMPARISON



KEY POINTS

- ✓ DataWeave is MuleSoft's transformation language.
- ✓ It helps you transform data between any formats.
- ✓ It is simple, powerful and highly performant.
- ✓ It improves productivity and reduces development time.
- ✓ It is the recommended way to do data transformations in Mule 4.

DATAWEAVE IN MULE FLOW



QUICK SYNTAX OVERVIEW

Concept	Syntax Example
Script Header	%dw 2.0
Output Directive	output json output xml output application/java
Payload Access	payload
Variable Declaration	var name = "John"
Object	{ key: value, key2: value2 }
Array	[1, 2, 3]
Function Call	upper("mule")

TOOLS THAT HELP

- API Designer
Design and validate DW scripts visually.
- RAML CLI
Validate, lint and bundle RAML files.
- VS Code Extension
RAML language support and snippets.
- CI/CD Pipelines
Automate validation and quality checks.

COMMON PITFALLS

- ⚠ Overusing includes making it hard to navigate.
- ⚠ Creating overly large RAML files.
- ⚠ Ignoring validation warnings and errors.
- ⚠ Inconsistent naming and formatting.
- ⚠ Not documenting error responses.

FINAL TAKEAWAY

- ★ Follow best practices to build RAML APIs that are consistent, secure, reusable, and easy to maintain.

Good RAML today,
Great APIs tomorrow!



Good RAML is not just about valid syntax – it's about **clarity**, **reuse** and **value** for your consumers. Follow best practices and build APIs that stand the test of time!



4.2 DATAWEAVE SYNTAX



DataWeave uses a simple and expressive syntax to transform data.

A DataWeave script is made of a **header** and a **body**.

SCRIPT STRUCTURE

```
%dw 2.0
output application/json
var x = 10
---
{
  message: "Hello",
  value: x
}
```

Header (Configuration)

Body (Transformation)

💡 Everything before --- is the header.
Everything after --- is the body.

SYNTAX OVERVIEW

Element	Description
%dw	Declares DataWeave version
output	Specifies the output data format
var	Declares a variable
---	Separates between header and body
{ }	Represents an object
[]	Represents an array
:	Key-value separator in objects
,	Separator between elements
()	Used in functions
" " or ' '	Used for string literals
//	Single line comment
/* */	Multi line comment

BASIC SYNTAX EXAMPLE

```
%dw 2.0
output application/json
---
{
  name: "John",
  age: 30,
  city: "New York"
}
```

✅ This script returns a JSON object with name, age and city.

HEADER DETAILS

- ✅ **%dw 2.0**
Declares DataWeave version 2.0
- ✅ **output <media-type>**
Defines the output format (json, xml, application/json, etc.)
- ✅ **var <name> = <value>**
Declares a variable
- ✅ **import * from dw::core::Strings**
Imports modules (optional)

Example:

```
%dw 2.0
output application/json
import * from dw::core::Strings
var greeting = "MuleSoft"
var name = "MuleSoft"
---
{ message: greeting ++ " " ++ name }
```

📘 Header configures how the script should behave.

BODY BASICS

- ✅ Contains the transformation logic
- ✅ Produces the output structure
- ✅ Can be an object { }, array [], value, or any expression
- ✅ Can use variables, functions, operators and keywords

Example:

```
%dw 2.0
output application/json
var name = "MuleSoft"
---
{
  upperName: upper(name),
  length: sizeof(name)
}
```

📘 Body defines what the output should look like.

DATA LITERALS (VALUES)

Literal Type	Syntax	Example	Result
String	"text"	"Hello"	Hello
Number	123	45.67	45.67
Boolean	true / false	true	true
Null	null	null	null
Array	[a, b, c]	[1, 2, 3]	Array of 3 items
Object	{ key: value }	{ a: 1, b: 2 }	Object with 2 keys

COMMENTS

Single line comment

```
%dw 2.0
output application/json
// This is a single line comment
var x = 10
---
x
```

Multi line comment

```
%dw 2.0
output application/json
/*
  This is a
  multi line comment
*/
---
"Done"
```

📘 Comments are ignored during execution.

COMMON OUTPUT DIRECTIVES

Directive	Description	Example
output application/json	Outputs JSON	output application/json
output application/xml	Outputs XML	output application/xml
output application/java	Outputs Java / POJO	output application/java
output text/csv	Outputs CSV	output text/csv
output application/yaml	Outputs YAML	output application/yaml

📘 If no output directive is specified, application/java is used by default.

VARIABLE DECLARATION

- ✅ Declared in the header using var
- ✅ Can use any expression as value
- ✅ Immutable (cannot be reassigned)
- ✅ Supports strings, numbers, arrays, objects and expressions

Example:

```
%dw 2.0
output application/json
var a = 100
var name = "MuleSoft"
var list = [1, 2, 3]
var obj = { x: 10, y: 20 }
---
{ a: a, name: name, list: list, obj: obj }
```

COMPLETE SYNTAX EXAMPLE

```
%dw 2.0
output application/json
import * from dw::core::Strings
// Variable declarations
var first = "Mule"
var last = "Soft"
var full = first ++ " " ++ last
var nums = [1, 2, 3, 4]
---
{
  fullName: full,
  upperName: upper(full),
  numbers: nums,
  count: sizeof(nums)
}
```

Result (JSON)

```
{
  "fullName": "MuleSoft",
  "upperName": "MULESOFT",
  "numbers": [1, 2, 3, 4],
  "count": 4
}
```

💡 Tip:
Use meaningful variable names and keep scripts simple and readable.

KEY TAKEAWAYS

- ★ A DataWeave script has two parts: Header and Body.
- ★ Header configures the script (version, output, variables, imports).
- ★ Body contains the transformation logic and returns the output.
- ★ DataWeave syntax is simple, expressive and easy to learn.
- ★ Use variables, functions and literals to build powerful transformations.
- ★ Comments and formatting improve readability.
- ★ Always test and keep your scripts clean and maintainable.



A clear syntax leads to clear transformations.
Write **simple**, **readable** and **reusable** DataWeave scripts!



4.3

DATA TYPES

MuleSoft

from salesforce

DataWeave supports a **rich** set of data types to represent and transform data.

Every value in DataWeave is an **expression** that has a specific data type.

OVERVIEW

✔ Data types define the kind of value a variable or expression can hold.

✔ DataWeave is strongly typed.

✔ Types help in validation, transformation and function usage.

✔ Data types can be primitive, complex or special.

CATEGORIES OF DATA TYPES

Primitive Types

- String
- Number
- Boolean
- Null
- Date
- DateTime

Basic single values

Complex Types

- Array
- Object

Collections of values

Special Types

- Binary
- Period
- Duration

Used for advanced scenarios

HOW TO CHECK TYPE

```
// Using typeOf() function
typeOf("Hello") // String
typeOf(123) // Number
typeOf(true) // Boolean
typeOf(null) // Null
typeOf([1, 2, 3]) // Array
typeOf({a: 1}) // Object
typeOf(2024-05-20) // Date
typeOf(2024-05-20T10:30:00Z) // DateTime
```

💡 **typeOf()** returns the data type of any value or expression.

DATAWEAVE DATA TYPES IN DETAIL

T

STRING

Sequence of characters enclosed in double quotes.

Example

"MuleSoft"

"Hello World"

Result

String

123

NUMBER

Represents numeric values. Supports integers and decimals.

Example

10

-10

45.67

3.1415

Result

Number

✔

BOOLEAN

Represents true or false value.

Example

true

false

Result

Boolean

⊘

NULL

Represents an empty or missing value.

Example

null

Result

Null

[]

ARRAY

Ordered collection of values. Elements can be of different types.

Example

[1, 2, "Mule", true]

["a", "b", "c"]

Result

Array

{ }

OBJECT

Unordered collection of key-value pairs.

Example

{

name: "John",

age: 30,

city: "New York"

}

Result

Object

📅

DATE

Represents date without time component.

Format: [YYYY-MM-DD]

Example

|2024-05-20|

Result

Date

🕒

DATETIME

Represents date with time and timezone.

Format: [YYYY-MM-DDTHH:mm:ssZ]

Example

|2024-05-20T10:15:30Z|

Result

DateTime

📄

BINARY

Represents binary data such as files, images and streams.

Example

write("Hello" as Binary)

Result

Binary

📅

PERIOD

Represents a period of time (date range).

Example

|P1Y2M10D|

|P3M|

Result

Period

🕒

DURATION

Represents a length of time (time duration).

Example

|PT1H30M|

|PT45M|

Result

Duration

TYPE CONVERSION (CASTING)

From	To	Example	Result
String	Number	("123" as Number)	123
String	Boolean	("true" as Boolean)	true
String	String	("abc" as String)	"abc"
String	Date	("2024-05-20" as Date)	2024-05-20
String	DateTime	("2024-05-20T10:30:00Z" as DateTime)	2024-05-20T10:30:00Z
Any	Binary	write(any as Binary)	Binary representation

💡 Use the as operator to convert values from one type to another.

TYPE COMPATIBILITY

Operation	Compatible Types
String Concatenation (+)	String + String
Arithmetic (+, -, *, /, %)	Number + Number
Comparison (=, !=, <, >, <=, >=)	Number, Date, DateTime, String
Logical (and, or, not)	Boolean
Array Functions	Array
Object Functions	Object

QUICK REFERENCE

String	"abc"
Number	10, -10, 45.67
Boolean	true, false
Null	null
Array	[1, 2, "a", true]
Object	{ name: "John" }
Date	2024-05-20
DateTime	2024-05-20T10:15:30Z
Binary	write("Hello" as Binary)
Period	P1Y2M10D (1y 2m 10d)
Duration	PT1H30M (1h 30m)

EXAMPLES IN DATAWEAVE SCRIPT

```
var id = 10
var name = "John" // String
var isActive = true // Boolean
var score = 98.5 // Number
var tags = ["api", "integration", "cloud"] // Array
var user = {
  id: id,
  name: name,
  created: |2024-05-20|,
  updated: |2024-05-20T10:15:30Z|
}
{ name, count: size(tags), active: isActive, user }
```

KEY POINTS

- ✔ DataWeave is strongly typed and every value has a specific data type.
- ✔ Use the right data type for your data to avoid errors.
- ✔ Types ensure validation and consistency.
- ✔ Objects store key-value pairs.
- ✔ Date, DateTime, Period and Duration handle time-related data.
- ✔ Use typeOf() to check the type of any value.
- ✔ Use as operator for explicit type conversion when required.

TIPS

- ✔ Combine multiple schemes if different clients need different methods.
- ✔ Always document how to obtain tokens or keys.
- ✔ Test security setups in API gateways and clients.
- ✔ Keep security schemes updated and reviewed.

Understanding data types is the foundation for writing correct and efficient DataWeave scripts.

4.4 VARIABLES



Variables in DataWeave store values that can be reused in your script. They make your transformations **readable**, **reusable** and **easy to maintain**.

OVERVIEW

- ✓ Variables store values for reuse.
- ✓ Declared in the header section using the `var` keyword.
- ✓ Improve readability and avoid repeating expressions.
- ✓ Can hold any DataWeave data type.
- ✓ Support scope: global (header) and local (inside functions/blocks).



Variables are reusable. Once assigned, their values can be used in the script.

SYNTAX

```
%dw 2.0
output application/json

var variableName = expression

// Use variableName in the body
{
  "result": variableName
}
```

WHERE TO DECLARE?

- ✓ Variables are declared in the header section, before the `----` separator.
- ✓ Multiple variables are separated by commas or new lines.

MULTIPLE VARIABLES

```
%dw 2.0
output application/json
var a = 10,
    b = "MuleSoft",
    c = [1, 2, 3]
----
{
  "a": a,
  "b": b,
  "c": c
}
```



Use meaningful names for variables to make scripts self-explanatory.

TYPES OF VARIABLES

GLOBAL VARIABLES (Header Scope)

- ✓ Declared in the header using `var`.
- ✓ Accessible throughout the entire script.
- ✓ Best for values used multiple times.

```
%dw 2.0
output application/json
var taxRate = 0.10
----
{
  "price": 100,
  "tax": price * taxRate,
  "total": 100 + (100 * taxRate)
}
```

LOCAL VARIABLES (Inside Functions/Blocks)

- ✓ Declared inside functions, objects or arrays.
- ✓ Accessible only within the local block.
- ✓ Best for intermediate or temporary values.

```
%dw 2.0
output application/json
----
(payload map ((item, index) -> do {
  var discounted = item.price * 0.9
  ----
  item ++ { price: discounted }
})))
```

CONSTANTS

- ✓ Use the `const` keyword to define constants.
- ✓ Helps in situations that a value will not change.

```
%dw 2.0
output application/json
const PI = 3.14159
----
{
  "pi": PI
}
```



const variables are also immutable.

EXAMPLES

Example 1: Simple Variable

```
%dw 2.0
output application/json
var name = "MuleSoft"
----
{
  "message": "Welcome " ++ name ++ "!"
}

Output
{ "message": "Welcome MuleSoft!" }
```

Example 2: Reusing Variables

```
%dw 2.0
output application/json
var price = 250,
    discount = 0.1
----
{
  "price": price,
  "discount": discount,
  "finalPrice": price - (price * discount)
}

Output
{ "price": 250, "discount": 0.1, "finalPrice": 225 }
```

Example 3: Using Expressions

```
%dw 2.0
output application/json
var items = [10, 20, 30],
    total = (items sum ((num) -> num))
----
{
  "items": items, "total": total
}

Output
{ "items": [10, 20, 30], "total": 60 }
```

Example 4: Local Variable in Function

```
%dw 2.0
output application/json
----
(payload map ((item) -> do {
  var tax = item.amount * 0.18
  ----
  item ++ { tax: tax }
})))

Output
Depends on the input payload
```

SCOPE RULES

Scope	Declared Where	Accessible Where	Example
Global	Header (before <code>----</code>)	Throughout the entire script	Configuration values, constants
Local	Inside functions, do blocks, objects	Only within that block	Intermediate calculations, temporary values

VARIABLE SHADOWING

A local variable with the same name as a global variable will override the global one inside the local block.

```
%dw 2.0
output application/json
var x = "global"
----
{
  "outer": x,
  "inside": (do { var x = "local" --- y })
}

Output
{ "outer": "global", "inside": "local" }
```

BEST PRACTICES

- ✓ Use meaningful and descriptive names.
- ✓ Declare variables at the top of the header.
- ✓ Use `const` for values that never change.
- ✓ Use `var` for values that should not change.
- ✓ Limit the scope of variables.
- ✓ Avoid too many variables if not required.



Good variable usage leads to clean, maintainable and efficient scripts.

COMMON MISTAKES

- ✗ Forgetting to declare variables in the header.
- ✗ Trying to modify a variable (variables are immutable).
- ✗ Using improper names such as `x`, `temp`.
- ✗ Declaring variables that are not used.

QUICK REFERENCE

Syntax	Example
Declare variable	<code>var x = 10</code>
Declare multiple variables	<code>var a = 1, b = 2</code>
Use variable	<code>x</code> or <code>a + b</code>
Local variable	<code>do { var y = 5 --- y }</code>

KEY TAKEAWAYS

- ★ Variables store values for reuse.
- ★ Declared in the header using `var` (or `const`).
- ★ Global variables are accessible throughout the script.
- ★ Local variables are accessible only within their scope.
- ★ Variables improve readability, reusability and maintainability.



Use variables wisely to write **clean**, **efficient** and **professional** DataWeave scripts.



4.5 OPERATORS

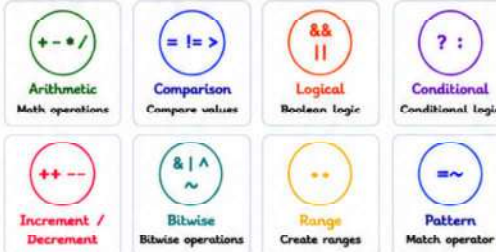


Operators in DataWeave are used to perform operations on values and expressions. They help you build **conditions**, **calculations** and **transformations** easily.

OVERVIEW

- Operators work on literals, variables, arrays, objects and expressions.
- DataWeave supports arithmetic, comparison, logical, conditional, string and other operators.
- Most operators return a value.
- Use parentheses () to control evaluation order.

OPERATOR CATEGORIES



BASIC SYNTAX EXAMPLE

```
%dw 2.0
output application/json
var a = 10
var b = 5
---
{
  sum: a + b,
  isGreater: a > b,
  isEqual: a == b
}
```

Operators make expressions powerful and easy to read.

ARITHMETIC OPERATORS

Operator	Name	Description	Example	Result
+	Addition	Adds two values	10 + 5	15
-	Subtraction	Subtracts right from left	10 - 5	5
*	Multiplication	Multiplies two values	10 * 5	50
/	Division	Divides left by right	10 / 5	2
%	Modulo	Returns remainder	10 % 3	1
^	Exponent	Power (left to the power right)	2 ^ 3	8
- (unary)	Unary Minus	Negates a value	-10	-10
+ (unary)	Unary Plus	Returns the positive value	+10	10

LOGICAL OPERATORS

Operator	Name	Description	Example	Result
&&	Logical AND	True if both are true	true && false	false
	Logical OR	True if any one is true	true false	true
!	Logical NOT	Inverts the value	!true	false

INCREMENT / DECREMENT OPERATORS

Operator	Name	Description	Example	Result
++	Increment	Increases value by 1	var x = 5; ++x	6
--	Decrement	Decreases value by 1	var x = 5; --x	4

STRING OPERATORS

Operator	Name	Description	Example	Result
++	Concatenation	Joins strings	"Hello" ++ "World"	"HelloWorld"
=~	Contains (Regex)	Checks regex match	"hello" =~ /he/	true
! =~	Not Contains (Regex)	Checks no regex match	"hello" ! =~ /he/	false

COMPARISON OPERATORS

Operator	Name	Description	Example	Result
==	Equal	Checks equality	10 == 10	true
!=	Not Equal	Checks inequality	10 != 5	true
>	Greater Than	Checks if left is greater	10 > 5	true
<	Less Than	Checks if left is less	10 < 5	false
>=	Greater or Equal	Checks if greater or equal	10 >= 5	true
<=	Less or Equal	Checks if less or equal	10 <= 5	false

CONDITIONAL (TERNARY) OPERATOR

Operator / Syntax	Description	Example	Result
condition ? valueIfTrue : valueIfFalse	Returns valueIfTrue if condition is true, otherwise valueIfFalse	10 > 5 ? "Yes" : "No"	"Yes"
		3 > 10 ? "Yes" : "No"	"No"

RANGE OPERATOR

Operator	Description	Example	Result
..	Creates a range from start to end (inclusive)	0 .. 5	[0, 1, 2, 3, 4, 5]
.. 			

OTHER USEFUL OPERATORS

Operator	Description	Example	Result
default	Returns default if value is null or empty	null default "N/A"	"N/A"
as	Casts a value to a specific data type	"123" as Number	123
in	Checks if value exists in an array	3 in [1, 2, 3, 4]	true
not in	Checks if value does not exist in an array	5 not in [1, 2, 3, 4]	true

OPERATORS PRECEDENCE (HIGH TO LOW)

Precedence	Category	Operators
1	Grouping	()
2	Unary	- + ++ -- (unary)
3	Exponent	^
4	Multiplication	* / %
5	Addition	+ -
6	Comparison	== != > < >= <=
7	Logical AND	&&
8	Logical OR	
9	Conditional	?:

EXAMPLES IN DATAWEAVE

```
%dw 2.0
output application/json
var a = 10
var b = 5
var name = "MuleSoft"
---
{
  arithmetic: { sum: a + b, diff: a - b, mul: a * b, mod: a % b },
  comparison: { greater: a > b, equal: a == b, lessOrEqual: a <= b },
  logical: { and: a > 5 && b < 10, or: a > 15 || b < 0, not: !false },
  conditional: a > b ? "a is greater" : "b is greater or equal",
  range: [1..5],
  concat: "Hello" ++ name
}
```

Output

```
{
  "arithmetic": { "sum": 15, "diff": 5, "mul": 50, "mod": 0 },
  "comparison": { "greater": true, "equal": false, "lessOrEqual": true },
  "logical": { "and": false, "or": true, "not": false },
  "conditional": "a is greater",
  "range": [1, 2, 3, 4, 5],
  "concat": "Hello MuleSoft"
}
```

BEST PRACTICES

- Use meaningful and descriptive names.
- Declare variables at the top of the header.
- Use const for values that never change.
- Use var for values that should not change.
- Limit the scope of variables.
- Avoid too many variables if not required.

Good variable usage leads to clean, maintainable and efficient scripts.

COMMON MISTAKES

- Forgetting operator precedence.
- Not using parentheses for complex conditions.
- Ignoring data types while comparing values.
- Not handling null cases in expressions.

SCOPE RULES

Scope	Declared Where	Accessible Where	Example
++ + / % +	Header (before ---)	Throughout the entire script	Configuration values, constants
== != > < >= <=	Comparison	Only within that block	Intermediate calculations, temporary values
&& !	Logical		
?:	Conditional (Ternary)		
..<	Range		
as	Membership		
default	Default Value		
in	Type Cast		

QUICK REFERENCE

Syntax	Example
Declare variable	var x = 10
Declare multiple variables	var a = 1, b = 2
Use variable	x or = + b
Local variable	do { var y = 5 --- } y

KEY TAKEAWAYS

- Variables store values for reuse.
- Declared in the header using var (or const).
- Global variables are accessible throughout the script.
- Local variables are accessible only within their scope.
- Variables improve readability, reusability and maintainability.



Mastering operators helps you write **efficient**, **readable** and **bug-free** DataWeave scripts.



4.6.1 CONDITIONAL STATEMENTS



Conditional statements in DataWeave allow you to make decisions and create different expressions based on conditions.

They are essential for building **dynamic** and **intelligent** transformations.

OVERVIEW

- ✓ Used to execute different logic based on conditions.
- ✓ Makes scripts dynamic and data-driven.
- ✓ Can be used with any expression that returns true or false.
- ✓ Frequently used in validation, mapping and filtering.

TYPES OF CONDITIONAL STATEMENTS

1. if-else



Simple condition with two paths.

2. else-if ladder



Multiple conditions checked in order.

3. when (switch)



Match a value against multiple cases.

4. default



Fallback case used inside when expressions.

SYNTAX OVERVIEW

Construct	Syntax
if-else	if (condition) do A else B
else-if	if (c1) do A else if (c2) do B else C
when	value1 -> A, value2 -> B
default	else C

1. IF - ELSE

Syntax

```
if (condition) do
  expression1
else
  expression2
```

Example

```
%dw 2.0
output application/json
var age = 20
---
if (age >= 18)
  "Adult"
else
  "Minor"
```

Output

```
{
  "status": "Adult"
}
```

2. ELSE IF LADDER

Syntax

```
if (condition1) do
  expr1
else if (condition2) do
  expr2
else if (condition3) do
  expr3
...
else
  exprN
```

Example

```
%dw 2.0
output application/json
var score = 75
---
if (score >= 90) "A"
else if (score >= 75) "B"
else if (score >= 50) "C"
else "F"
```

Output

```
{
  "grade": "B"
}
```



Use **if-else** and **else-if** statements whenever your transformation logic depends on one or more conditions.

4.6.2 CONDITIONAL STATEMENTS (Part 2)



3. WHEN (SWITCH)

Syntax

```
when
value case1 -> result1
value case2 -> result2
...
else defaultResult
```

Example

```
%dw 2.0
output application/json
var day = "Mon"
---
when day
  "Mon", "Tue", "Wed", "Thu", "Fri"
  -> "Weekday"
  "Sat", "Sun" -> "Weekend"
  else "Invalid Day"
```

Output

```
{ "type" : "Weekday" }
```

4. DEFAULT (IN WHEN)

Syntax

```
when
value case1 -> result1
value case2 -> result2
...
else defaultResult
```

Example

```
%dw 2.0
output application/json
var role = "Guest"
---
when role
  "Admin" -> "Full Access"
  "User" -> "Limited Access"
  else "Read Only"
```

Output

```
{ "access" : "Read Only" }
```

5. NESTED CONDITIONALS

Example

```
%dw 2.0
output application/json
var customer = { type: "Premium", active: true }
---
if (customer.active) do
  if (customer.type == "Premium") do
    "Welcome Premium Customer"
  else "Welcome Customer"
  else "Inactive Customer"
```

Output

```
{ "message" : "Welcome Premium Customer" }
```

6. CONDITIONALS WITH BOOLEANS

Example

```
%dw 2.0
output application/json
var isMember = true
---
if (isMember) do "Member: true and valid"
else false
```

Output

```
{ "isMemberEligible" : false }
```

7. CONDITIONALS WITH NULL CHECK

Example

```
%dw 2.0
output application/json
var name = null
---
if (isEmpty(name) or name == null)
  do "Guest"
else name
```

Output

```
{ "displayName" : "Guest" }
```

PRACTICAL SCENARIOS

Scenario	Example Use Case
Data Validation	Check if required fields are present.
Mapping Values	Map codes to meaningful descriptions.
Business Rules	Apply different logic based on conditions.
Status Calculation	Calculate status based on multiple criteria.
Error Handling	Provide default values or error messages.

BEST PRACTICES

- ✓ Keep conditions simple and readable.
- ✓ Use meaningful variable and field names.
- ✓ Prefer when for multiple discrete values.
- ✓ Handle all possible cases using else or default.
- ✓ Use isEmpty, isBlank or null to check for null/empty.
- ✓ Test all branches to ensure expected results.

COMMON MISTAKES

- ✗ Forgetting else case to handle no match.
- ✗ Using = instead of == for equality.
- ✗ Ignoring data types while comparing values.
- ✗ Not checking for null or empty values.
- ✗ Returning inconsistent data types in branches.



Good use of conditional statements leads to **smart**, **flexible** and **maintainable** DataWeave scripts.

4.6.3 CONDITIONAL STATEMENTS (Part 3)



OPERATORS PRECEDENCE (HIGH TO LOW)

Precedence	Category	Operators	Example
1	Grouping	()	(a + b) * c
2	Unary	!, +, -, ++ (unary)	-a, !flag
3	Exponent	^	a ^ 2
4	Multiplicative	*, /, %	a * b, a % b
5	Additive	+, -	a + b, a - b
6	Comparison	==, !=, >, <, >=, <=	a >= b
7	Logical AND	&&	a > 0 && b > 0
8	Logical OR		a > 0 b > 0
9	Conditional	?:	condition ? x : y



Operators with higher precedence are evaluated first.
Use parentheses () to control evaluation order.

SCOPE RULES

Scope	Declared Where	Accessible Where	Example
Global	Header (before ---)	Throughout the entire script	var x = 10 // accessible everywhere
Local	Inside functions, do blocks, objects	Only within that block	do { var y = 5 --- y }
Default Value	As variable initialization	Where the variable is used	var n = 0 default 10
Type Cast	During initialization	Where the variable is used	var age = "20" as Number // age is number



Local variables cannot be accessed outside their scope.
Global variables are accessible across the entire script.

QUICK REFERENCE

Syntax	Example
if - else	if (condition) do A else B
else-if ladder	if (c1) do A else if (c2) do B else C
when (switch)	value1 -> A value2 -> B else C
default (in when)	else C
Nested conditions	if (cond1) do if (cond2) do X else Y else Z
Boolean operators	&& (AND), (OR), ! (NOT)
Comparison operators	==, !=, >, <, >=, <=
Null check	isEmpty(value) or value == null

COMMON MISTAKES

- ✗ Forgetting else case in if-else or else-if ladder.
- ✗ Using = instead of == for equality comparison.
- ✗ Ignoring data types while comparing values.
- ✗ Not checking for null or empty values.
- ✗ Returning inconsistent data types in different branches.
- ✗ Overly complex conditions - break into smaller parts.
- ✗ Not using parentheses, leading to wrong results.

EXAMPLES IN DATAWEAVE

```
%dw 2.0
output application/json
var a = 10
var b = 5
var name = "MuleSoft"
---
{
  "arithmetic": { "sum": a + b, "diff": a - b,           // 15, 5, 50, 2, 0
                  "mul": a / b, "mod": a % b },
  "comparison": { "greater": a > b, "equal": a == b,     // true, false, false
                  "lessOrEqual": a <= b },
  "logical": { "and": (a > 5 && b > 2), "or": (a >       // true, false, true
              '> 15 || b < 0), "not": !(a == b) },
  "conditional": "a is greater"                       // ternary operator
                ((a > b) ? "a is greater" : "b is greater or equal"),
  "range": [1, 2, 3, 4, 5],
  "concat": "Hello " ++ name                          // string concatenation
}
```

Output

```
{
  "arithmetic": { "sum": 15, "diff": 5, "div": 2, "mod": 0 },
  "comparison": { "greater": true, "equal": false, "lessOrEqual": false
  "logical": { "and": true, "or": false, "not": true },
  "conditional": "a is greater",
  "range": [1, 2, 3, 4, 5],
  "concat": "Hello MuleSoft"
}
```

KEY TAKEAWAYS

- ★ Conditional statements help make decisions in your DataWeave scripts.
- ★ Use if-else for simple conditions and else-if for multiple conditions.
- ★ Use when (switch) for matching a value against multiple cases.
- ★ Always handle unmatched cases using default or else.
- ★ Use parentheses to control evaluation order and avoid errors.
- ★ Check for null or empty values to prevent runtime issues.
- ★ Keep conditions simple, readable and well-structured.
- ★ Well-written conditions make scripts robust, maintainable and efficient.

BEST PRACTICES

- ✓ Use meaningful variable and field names.
- ✓ Keep conditions short and easy to understand.
- ✓ Prefer early return / default to reduce nesting.
- ✓ Validate input data before applying conditions.
- ✓ Use functions like isEmpty(), isBlank() and default for safety.
- ✓ Test all possible branches with sample data.
- ✓ Comment complex conditions for future reference.



Use conditional statements to write **smart**, **flexible** and **maintainable** DataWeave scripts.

4.7.1

ARRAY FUNCTIONS (Part 1)

 MuleSoft
from  salesforce

- Array functions in DataWeave help you process, transform and analyze array data easily.
- They are powerful, expressive and can be combined to solve complex scenarios.

OVERVIEW

- ✓ DataWeave provides rich array functions.
- ✓ Functions are immutable and return new arrays.
- ✓ Functions can be chained together.
- ✓ Support mapping, filtering, reducing and grouping.
- ✓ Useful in real-time integration transformations.

COMMON ARRAY FUNCTIONS

Function	Purpose
 <code>map()</code>	Transform each element
 <code>filter()</code>	Filter elements
 <code>reduce()</code>	Reduce array to single value
 <code>pluck()</code>	Extract field values
 <code>flatten()</code>	Flatten nested arrays
 <code>distinctBy()</code>	Remove duplicates based on key
 <code>groupBy()</code>	Group by key
 <code>orderBy()</code>	Sort values

SYNTAX OVERVIEW

<code>map()</code>	array map ((item,index) -> expression)
<code>filter()</code>	array filter ((item,index) -> condition)
<code>reduce()</code>	array reduce ((item,acc,index) -> expression)
<code>pluck()</code>	array pluck(key)
<code>flatten()</code>	array flatten()
<code>distinctBy()</code>	array distinctBy((item) -> key)
<code>groupBy()</code>	array groupBy((item) -> key)
<code>orderBy()</code>	array orderBy((item) -> key)

1. map()

Description

Transforms every element using an expression.

Example

```
var nums = [1, 2, 3, 4]
---
nums map ((n) -> n * 2)
```

Output

[2, 4, 6, 8]

2. filter()

Description

Returns elements matching a condition.

Example

```
var nums = [1, 2, 3, 4, 5, 6]
---
nums filter ((n) -> n mod 2 == 0)
```

Output

[2, 4, 6]

3. reduce()

Description

Reduces an array into a single value.

Example

```
var nums = [1, 2, 3, 4]
---
nums reduce ((acc, n) -> acc + n)
```

Output

10



`map()`, `filter()` and `reduce()` are the three most commonly used array functions in DataWeave and are heavily used in MuleSoft projects.

4.7.3

ARRAY FUNCTIONS (Part 3)



OPERATORS PRECEDENCE (HIGH TO LOW)

Precedence	Category	Operators	Example
1	Grouping	()	(a + b) * c
2	Unary	!, ~, -, ++ (unary)	~a, !flag
3	Exponent	^	a ^ 2
4	Multiplicative	*, /, %	a * b, a % b
5	Additive	+, -	a + b, a - b
6	Comparison	==, !=, >, <, >=, <=	a >= b
7	Logical AND	&&	a > 0 && b > 0
8	Logical OR		a > 0 b > 0
9	Conditional	?:	condition ? x : y



Operators with higher precedence are evaluated first.
Use parentheses () to control evaluation order.

SCOPE RULES

Scope	Declared Where	Accessible Where	Example
Global	Header (before ---)	Throughout the entire script	var x = 10 // accessible everywhere
Local	Inside functions, do blocks, objects	Only within that block	do { var y = 5 --- y }
Default Value	As variable initialization	Where the variable is used	var n = 0 default 10
Type Cast	During initialization	Where the variable is used	var age = "20" as Number // age is number



Local variables cannot be accessed outside their scope.
Global variables are accessible across the entire script.

QUICK REFERENCE

Syntax	Example
map()	var a = [1, 2, 3] map (\$+2)
filter()	var a = [1, 2, 3] filter (\$ > 1)
reduce()	var a = [1, 2, 3] reduce ((acc, n) => acc + n, 0)
pluck()	var a = [{ "n": 1 }, { "n": 2 }] pluck ("n")
flatten()	var a = [[1], [2]] flatten()
distinctBy()	var a = [{id: 1}, {id: 1}] distinctBy \$.id
groupBy()	var a = [{c: "A"}, {c: "B"}, {c: "A"}] groupBy \$.c
orderBy()	var a = [{s: 30}, {s: 10}] orderBy \$.s
flatten()	var a = [[1], [2]] flatten()

COMMON MISTAKES

- ✗ Using map() for filtering (use filter() instead).
- ✗ Forgetting to return accumulator in reduce().
- ✗ Using distinctBy() without proper key selector.
- ✗ Sorting without considering data types.
- ✗ Over-nesting functions makes script hard to read.
- ✗ Not handling empty arrays in transformations.

EXAMPLES IN DATAWEAVE

```
%dw 2.0
output application/json
var a = 10
var b = 5
var name = "MuleSoft"

---
{
  "arithmetic": { "sum": a + b, "diff": a - b,           // 15, 5, 50, 2, 0
                  "mul": a * b, "mod": a % b },
  "comparison": { "greater": a > b, "equal": a == b;     // true, false, false
                  "lessOrEqual": a <= b },
  "logical": { "and": (a > 5 && b > 2), "or": (a > 7),   // true, false, true
               "not": !(a == b) },
  "conditional": "a is greater",                       // ternary operator
                 ((a > b) ? "a is greater" : "b is greater or equal"),
  "range": [1, 2, 3, 4, 5],
  "concat": "Hello " ++ name                          // string concatenation
}
```

Output

```
{
  "arithmetic": { "sum": 15, "diff": 5, "mul": 50, "mod": 0 },
  "comparison": { "greater": true, "equal": false, "lessOrEqual": false },
  "logical": { "and": true, "or": true, "not": true },
  "conditional": "a is greater",
  "range": [1, 2, 3, 4, 5],
  "concat": "Hello MuleSoft"
}
```

KEY TAKEAWAYS

- ★ Array functions are the backbone of DataWeave transformations.
- ★ They help you filter, transform, aggregate and organize array data.
- ★ Combine multiple functions to solve complex business problems.
- ★ Functions are immutable and return new arrays.
- ★ Chaining functions makes scripts clean, readable and efficient.
- ★ Leverage functions to improve performance and maintainability.

BEST PRACTICES

- ✔ Always use the right function for the task.
- ✔ Prefer filter() before map() to reduce unnecessary processing.
- ✔ Use distinctBy() to remove duplicates by a key.
- ✔ Use orderBy() to sort data for reporting and analysis.
- ✔ Chain functions for expressive and concise transformations.
- ✔ Avoid heavy processing in large arrays when performance matters.



Master array functions to write **concise**, **powerful** and **performant** DataWeave transformations!



4.8.1 OBJECT FUNCTIONS

- Object functions in DataWeave help you **transform**, **extract** and **analyze** data stored in objects (key-value pairs).
- They make your scripts **cleaner**, **reusable** and more **powerful**.

OVERVIEW

- Objects in DataWeave are collections of key-value pairs.
- Object functions help manipulate and transform object data.
- These functions return new objects, arrays or values.
- Widely used in mapping, filtering and data transformations.

COMMON OBJECT FUNCTIONS

Function	Purpose
<code>mapObject()</code>	Transform each key-value pair
<code>pluck()</code>	Extract values from keys
<code>keysOf()</code>	Get all object keys
<code>valuesOf()</code>	Get all object values
<code>namesOf()</code>	Get names of all keys

SYNTAX OVERVIEW

<code>mapObject()</code>	object <code>mapObject((value, key, index) -> expression)</code>
<code>pluck()</code>	object <code>pluck(key)</code>
<code>keysOf()</code>	object <code>keysOf()</code>
<code>valuesOf()</code>	object <code>valuesOf()</code>
<code>namesOf()</code>	object <code>namesOf()</code>

★ All object functions are immutable. They never modify the original object.

1. `mapObject()`

- Applies an expression to every key-value pair.
- Returns a new transformed object.

Example:

```
var obj = {
  name : "John",
  age  : 30,
  city : "NY"
}
---
obj mapObject((value, key, index) ->
  {(key): upper(value as String)})
)
```

Output:

```
{
  "name" : "JOHN",
  "age"  : "30",
  "city" : "NY"
}
```

2. `pluck()`

- Extracts values from one or more object keys.
- Returns an array.

Example:

```
var obj = {
  name : "John",
  age  : 30,
  city : "NY"
}
---
obj pluck ["name", "city"]
```

Output:

```
[
  "John",
  "NY"
]
```



`mapObject()` and `pluck()` are among the most frequently used object functions in MuleSoft DataWeave transformations.

4.8.2 OBJECT FUNCTIONS (Part 2)



★ Object functions help you **extract information**, **get keys** and **values**, and work with object data efficiently.

3. keysOf()

- Returns an array of all keys in the object.
- Useful to get the list of keys.

Example:

```
var obj = {
  name : "John",
  age  : 30,
  city : "NY"
}
---
obj keysOf()
```

Output:

```
[ "name", "age", "city" ]
```

4. valuesOf()

- Returns an array of all values in the object.
- Useful to get all values together.

Example:

```
var obj = {
  name : "John",
  age  : 30,
  city : "NY"
}
---
obj valuesOf()
```

Output:

```
[ "John", 30, "NY" ]
```

5. namesOf()

- Returns an array of key names.
- Similar to keysOf(), used for clarity in some scenarios.

Example:

```
var obj = {
  name : "John",
  age  : 30,
  city : "NY"
}
---
obj namesOf()
```

Output:

```
[ "name", "age", "city" ]
```

USING FUNCTIONS WITH COMPLEX OBJECT

Example:

```
var employee = {
  id   : 101,
  name : "Alice",
  details : {
    role   : "Architect",
    salary : 75000,
    location : "Bangalore"
  },
  skills : ["DW", "Mule", "Java"]
}
---
employee keysOf()
employee valuesOf()
```

Output:

```
[ "id", "name", "details", "skills" ]
[ 101, "Alice", { ... }, [ "DW", "Mule", "Java" ] ]
```

NESTED OBJECT SCENARIO

Example:

```
var data = {
  order : { id : 1, name : "Book",
            active : true },
  customer : { name : "Bob",
              balance : 150 }
}
---
var orderInfo = data.order pluck ["id","name"]
var customerInfo = data.customer valuesOf()
var allKeys = data keysOf()
```

Output:

```
orderInfo   : { "id": 1, "name": "Book" }
customerInfo : [ "Bob", 150 ]
allKeys      : [ "order", "customer" ]
```

COMBINING FUNCTIONS

Example:

```
var obj = {
  a : 10, b : 20, c : 30
}
---
// Get keys and then get values
(obj keysOf())
  map ((k) -> obj[k])
```

Output:

```
[ 10, 20, 30 ]
```

BEST PRACTICES

- ✓ Use keysOf() to iterate over object keys.
- ✓ Use valuesOf() when you need all values.
- ✓ Use pluck() to fetch specific fields as an array.
- ✓ Combine functions (e.g., keysOf() + map()) for advanced transformations.
- ✓ Always handle nested objects carefully.

COMMON MISTAKES

- ✗ Using pluck() with wrong key name.
- ✗ Assuming order of keys in objects (objects are unordered).
- ✗ Forgetting that object functions return new results.
- ✗ Not handling null values in object fields.
- ✗ Overusing functions instead of simple mapping.



Use object functions wisely to write **clean**, **efficient** and **powerful** DataWeave transformations!



4.8.3 OBJECT FUNCTIONS (Part 3)



Object functions are **powerful** tools in DataWeave.

They can be **chained** and **combined** to solve complex transformation requirements.

KEY TAKEAWAYS

- ★ Object functions help transform, extract and analyze object data.
- ★ They return new objects, arrays or values.
- ★ These functions are immutable and do not modify the original object.
- ★ Functions can be chained together for powerful transformations.

FUNCTIONS RECAP

Function	Purpose
mapObject()	Transform each key-value pair
pluck()	Extract values from keys
keysOf()	Get all object keys
valuesOf()	Get all object values
namesOf()	Get names of all keys

CHEAT SHEET (SYNTAX)

mapObject()	object mapObject((value, key, index) -> expression)
pluck()	object pluck(key)
keysOf()	object keysOf()
valuesOf()	object valuesOf()
namesOf()	object namesOf()

ADVANCED USAGE OF OBJECT FUNCTIONS

1. Rename all keys using mapObject()

Example:

```
var obj = {
  firstName: "John",
  lastName: "Doe",
  age: 30
}
---
obj mapObject((value, key, index) ->
  {(upper(key)): value} )
```

Output:

```
{
  FIRSTNAME: "John",
  LASTNAME: "Doe",
  AGE: 30
}
```

2. Filter object by keys and values

Example:

```
var obj = {
  name: "John",
  age: 30,
  city: "NY",
  role: "Developer"
}
---
obj filterObject ((value, key, index) ->
  (key != "age" and value != "NY" ) )
```

Output:

```
{
  name: "John",
  role: "Developer"
}
```

3. Create object from keys and values

Example:

```
var keys = ["id", "name", "city"]
var values = [101, "Alice", "Boston"]
---
keys zip values
reduce ((acc, item, index) ->
  acc ++ {(item[0]): item[1]}, {})
```

Output:

```
{
  id: 101,
  name: "Alice",
  city: "Boston"
}
```

REAL-WORLD SCENARIO

Transform customer data object to return only required fields with formatted values.

Example:

```
var customer = {
  id: 501,
  name: "Alice Johnson",
  email: "alice@example.com",
  city: "new york",
  joined: "2024-01-15",
  active: true
}
---
customer
filterObject ((value, key) -> key != "active")
mapObject ((value, key) ->
  if (key == "city") {(key): upper(value)}
  else {(key): value})
```

Output:

```
{
  id: 501,
  name: "Alice Johnson",
  email: "alice@example.com",
  city: "NEW YORK",
  joined: "2024-01-15"
}
```



We filtered out "active" field and converted city value to uppercase using mapObject().

COMBINING MULTIPLE FUNCTIONS

Example:

```
var obj = { a: 10, b: 20, c: 30, d: 40 }
---
obj
mapObject ((value, key) -> {(key): value * 2})
filterObject ((value, key) -> value >= 40)
pluck (keysOf($)[0]) // get first key's value
```

Output:

80

BEST PRACTICES

- ✓ Use keysOf() and valuesOf() to explore object data.
- ✓ Use mapObject() for conditional formatting and renaming.
- ✓ Chain functions for concise and powerful transformations.
- ✓ Always handle nullable or empty objects safely.
- ✓ Write reusable functions for common object operations.

COMMON MISTAKES

- ✗ Modifying original objects (Functions return new objects).
- ✗ Using pluck() with wrong or non-existing key.
- ✗ Not handling missing keys in objects.
- ✗ Overusing mapObject() when simple pluck() is enough.
- ✗ Assuming order in objects (objects are unordered).



Master object functions to write **clean**, **efficient** and **powerful** DataWeave transformations!



4.9.1 STRING FUNCTIONS (Part 1)



String functions in DataWeave help you **manipulate, transform** and work with text data efficiently.

They are widely used for **data cleaning**, formatting and integration tasks.

OVERVIEW

- ✓ Strings are sequences of characters enclosed in quotes.
- ✓ DataWeave provides rich and powerful functions to work with text.
- ✓ Helps in cleaning, validation, formatting and data preparation.
- ✓ Essential in real-world integration and API payloads.

COMMON STRING FUNCTIONS

ABC upper() Convert to uppercase	abc lower() Convert to lowercase	 trim() Remove leading and trailing whitespace	123 sizeOf() Returns length of the string
 splitBy() Split string into an array	 joinBy() Join array of strings	 replace() Replace text or pattern	 contains() Check if string contains text

SYNTAX OVERVIEW

Function	Syntax
upper	upper(text)
lower	lower(text)
trim	trim(text)
sizeOf	sizeOf(text)
splitBy	splitBy(text, separator)
joinBy	joinBy(array, separator)
replace	replace(text, oldText, newText)
substring	substring(text, fromIndex[, toIndex])
contains	contains(text, substring)
startsWith	startsWith(text, prefix)
endsWith	endsWith(text, suffix)
matches	matches(text, regex)

1. upper()

Converts all characters to uppercase.

Example

```
var str = "MuleSoft Rocks"
---
upper(str)
```

Output

"MULESOFT ROCKS"

2. lower()

Converts all characters to lowercase.

Example

```
var str = "MULESOFT ROCKS"
---
lower(str)
```

Output

"mulesoft rocks"

3. trim()

Removes leading and trailing whitespace.

Example

```
var str = " Hello World "
---
trim(str)
```

Output

"Hello World"

4. sizeOf()

Returns the number of characters in the string.

Example

```
var str = "DataWeave 2.0"
---
sizeOf(str)
```

Output

12

5. splitBy()

Splits the string into an array using the given separator.

Example

```
var str = "a,b,c,d"
---
splitBy(str, ",")
```

Output

["a", "b", "c", "d"]



These are some of the most commonly used string functions in **DataWeave** for **text manipulation**!

4.9.2 STRING FUNCTIONS (Part 2)



MORE USEFUL STRING FUNCTIONS

⑥ joinBy()

Joins an array of strings using the given separator.

Example:

```
var arr = ["Jan", "Feb", "Mar"]
---
joinBy(arr, " | ")
```

Output:

"Jan | Feb | Mar"

⑦ replace()

Replaces all occurrences of a pattern with another string.

Example:

```
var str = "I love MuleSoft"
---
replace(str, "MuleSoft",
        "DataWeave")
```

Output:

"I love DataWeave"

⑧ substring()

Extracts a substring using start index and length (or end index).

Example:

```
var str = "Hello DataWeave"
---
substring(str, 0, 5)
```

Output:

"Hello"

⑨ contains()

Checks if a string contains the specified substring.

Example:

```
var str = "Welcome to MuleSoft"
---
contains(str, "MuleSoft")
```

Output:

true

⑩ startsWith()

Checks if string starts/ends with the given prefix.

Example:

```
var str = "Hello World"
---
startsWith(str, "Hello")
```

Output:

true

⑪ endsWith()

Checks if string starts/ends with the given suffix.

Example:

```
var str = "Hello World"
---
endsWith(str, "World")
```

Output:

true



REAL-WORLD SCENARIOS

Scenario	Function(s) Used	Description
Standardize text	upper(), lower()	Normalize text format
Extract names	splitBy(), trim()	Split and clean text to get names
Calculate length	sizeOf()	Get total number of characters
Clean text	trim(), replace()	Remove spaces and unwanted text
Remove duplicates	distinctBy()	Eliminate duplicate records
Join values	joinBy()	Combine array elements
Format reports	replace(), padLeft()	Format text for reports/tables



BEST PRACTICES

- ✓ Use trim() to clean user inputs before validations.
- ✓ Prefer splitBy() over complex regex for parsing simple data.
- ✓ Use replace() with regex carefully for complex text patterns.
- ✓ Avoid hardcoding text; use variables or lookups where possible.
- ✓ Use startsWith() / endsWith() for prefix/suffix validations.
- ✓ Avoid heavy processing in large strings when performance matters.



Use the **right string function** at the right time to write **clean**, **efficient** and **powerful DataWeave** scripts!

4.9.3 STRING FUNCTIONS (Part 3)



COMMON MISTAKES

- ❌ 1. **Forgetting strings are immutable.**
String functions return new strings.
They do not change the original string.
- ❌ 2. **Not handling null values.**
Always check for null before
applying string functions.
- ❌ 3. **Overusing replace().**
Using replace() too often can make
scripts hard to read and maintain.
- ❌ 4. **Assuming contains() is case-sensitive.**
contains() is case-sensitive by default.
Use lower()/upper() for case-insensitive search.
- ❌ 5. **Modifying original strings.**
Do not try to change strings directly.
Always store the result in a new variable.

QUICK REFERENCE TABLE

Function	Purpose
upper()	Convert text to uppercase
lower()	Convert text to lowercase
trim()	Remove leading and trailing spaces
sizeOf()	Return length of the string
splitBy()	Split string into an array
joinBy()	Join array of strings
replace()	Replace text or pattern
substring()	Extract part of the string
contains()	Check if string contains text
startsWith()	Check if string starts with prefix
endsWith()	Check if string ends with suffix
matches()	Check if string matches regex

SYNTAX CHEAT SHEET

Function	Syntax	Function	Syntax
upper(text)	upper(text)	replace(text, oldText, newText)	replace(text, oldText, newText)
lower(text)	lower(text)	substring(text, fromIndex, toIndex)	substring(text, fromIndex, toIndex)
trim(text)	trim(text)	contains(text, substring)	contains(text, substring)
sizeOf(text)	sizeOf(text)	startsWith(text, prefix)	startsWith(text, prefix)
splitBy(text, separator)	splitBy(text, separator)	endsWith(text, suffix)	endsWith(text, suffix)
joinBy(array, separator)	joinBy(array, separator)	matches(text, regex)	matches(text, regex)

KEY TAKEAWAYS

- ★ String functions help you **extract**, **transform** and **manipulate** text data efficiently.
- ★ They return new strings and are **immutable**.
- ★ Use the right function for the task to keep your scripts **clean** and **readable**.
- ★ **Combine** multiple functions to solve complex transformation requirements.
- ★ Write **clean**, **efficient** and **maintainable** DataWeave scripts using string functions.

EXAMPLE COMBO

```
var str = " Hello MuleSoft "
---
trim(str)           // "Hello MuleSoft"
lower(trim(str))    // "hello mulesoft"
upper(trim(str))    // "HELLO MULESOFT"
sizeOf(trim(str))   // 15
splitBy(trim(str), " ") // ["Hello", "MuleSoft"]
replace(trim(str), "MuleSoft", "World") // "Hello World"
```



Master string functions to write **clean**,
efficient and **maintainable** DataWeave transformations!

4.10.1

DATE & TIME FUNCTIONS

 MuleSoft
from salesforce

★ Date & Time functions in DataWeave help you **create**, **parse**, **format**, **manipulate** and **compare** dates and times. They are essential for handling **time zones**, **durations** and **real-world integrations**.

OVERVIEW

- ✓ DataWeave uses Java Time API under the hood.
- ✓ All date and time values are **immutable**.
- ✓ Supports parsing, formatting, arithmetic, comparison and **time zone handling**.
- ✓ Key concepts include:
 - Date
 - LocalTime
 - Time
 - TimeZone
 - DateTime
 - Duration
 - LocalDate
 - Period

COMMON DATE & TIME FUNCTIONS

	<code>now()</code>	Current date and time
	<code>today()</code>	Current date
	<code>format()</code>	Format date/time
	<code>parse()</code>	Parse string into date/time
	<code>add()</code>	Add duration/period
	<code>subtract()</code>	Subtract duration/period
	<code>withZone()</code>	Convert time zone
	<code>between()</code>	Difference between dates
	<code>isBefore()</code>	Compare dates (before)
	<code>isAfter()</code>	Compare dates (after)
	<code>atZone()</code>	Attach time zone to date/time

SYNTAX OVERVIEW

- `now()`
- `format(dateTime, pattern)`
- `parse(string, pattern)`
- `add(dateTime, duration/period)`
- `subtract(dateTime, duration/period)`
- `withZone(dateTime, zone)`
- `between(date1, date2, unit)`
- `isBefore(date1, date2)`
- `isAfter(date1, date2)`
- `atZone(localDateTime, zone)`

 All Date & Time functions do not modify the original object. They return new results.

1. `now()`

Returns the current date & time in ISO-8601 format.

Example :

```
%dw 2.0
output application/json
---
now()
```

Output (example):

2024-05-20T14:35:22.123+04:00

2. `today()`

Returns the current date without the time portion.

Example :

```
%dw 2.0
output application/json
---
today()
```

Output (example):

2024-05-20

3. `format()`

Formats a date or datetime using a specified pattern.

Example :

```
%dw 2.0
output application/json
var dt = "2024-05-20T14:35:22Z"
---
format(dt, "yyyy-MM-dd HH:mm")
```

Output :

2024-05-20 14:35

4. `parse()`

Parses a string into a Date or DateTime value.

Example :

```
%dw 2.0
output application/json
var dt = "2024-05-20 14:35:22"
---
parse(dt, "yyyy-MM-dd HH:mm:ss")
```

Output :

2024-05-20T14:35:22

★ The most frequently used Date & Time functions in MuleSoft projects are `now()`, `today()`, `format()` and `parse()`. 

4.10.2

DATE & TIME FUNCTIONS

 MuleSoft
from  salesforce



Advanced Date & Time functions help you perform **arithmetic**, **time zone conversions**, **comparisons** and **duration calculations**.

5. add()

Adds a duration or period to a date/time.

Example :

```
%dw 2.0
output application/json
var dt = "2024-05-20T10:00:00Z"
---
add(dt, |P1D|)
```

Output :

2024-05-21T10:00:00Z

6. subtract()

Subtracts a duration or period from a date/time.

Example :

```
%dw 2.0
output application/json
var dt = "2024-05-20T10:00:00Z"
---
subtract(dt, |PT5H|)
```

Output :

2024-05-20T05:00:00Z

7. withZone()

Converts a date/time to a different time zone.

Example :

```
%dw 2.0
output application/json
var dt = "2024-05-20T10:00:00Z"
---
withZone(dt, "America/New_York")
```

Output :

2024-05-20T06:00:00-04:00

8. between()

Returns the difference between two date/times in the given unit.

Example :

```
%dw 2.0
output application/json
var d1 = "2024-05-20"
var d2 = "2024-05-25"
---
between(d1, d2, "days")
```

Output :

5

9. isBefore() / isAfter()

Compares two date/times.

Example :

```
%dw 2.0
output application/json
var d1 = "2024-05-20"
var d2 = "2024-05-25"
---
{
  before : isBefore(d1, d2),
  after  : isAfter(d1, d2)
}
```

Output :

{ before: true, after: false }

10. atZone()

Attaches a time zone to a local DateTime (without zone).

Example :

```
%dw 2.0
output application/json
var localDt = "2024-05-20T10:00:00"
---
atZone(localDt, "Asia/Kolkata")
```

Output :

2024-05-20T10:00:00+05:30"



These functions help you perform powerful date & time operations efficiently and build real-world, **time-aware** solutions.

DATE & TIME UNITS FOR between()

Unit	Description
years	Difference in years
months	Difference in months
days	Difference in days
hours	Difference in hours
minutes	Difference in minutes
seconds	Difference in seconds

DURATION & PERIOD EXAMPLES

Format	Example
P1D	// 1 day
PT5H	// 5 hours
PT30M	// 30 minutes
P7D	// 7 days
P1M	// 1 month
P1Y	// 1 year
P1DT2H	// 1 day 2 hours
P2DT3H	// 2 days 3 hours

COMMON TIME ZONES

Time Zone ID	Description
UTC	Coordinated Universal Time
America/New_York	Eastern Time (US)
Europe/London	London Time
Asia/Kolkata	India Standard Time
Asia/Tokyo	Japan Standard Time
Australia/Sydney	Australian Eastern Time

DATE & TIME SCENARIOS

- ✔ Log timestamps in UTC and convert at the edges.
- ✔ Calculate SLAs and timeouts.
- ✔ Handle user time zones for reports and notifications.
- ✔ Schedule jobs and reminders.
- ✔ Validate data within specific date ranges.

COMMON MISTAKES

- ✗ Parsing without specifying the correct pattern.
- ✗ Mixing up Duration and Period.
- ✗ Ignoring time zones and using server local time.
- ✗ Assuming between() returns fractional values.
- ✗ Not handling null or invalid date strings.



Leverage Date & Time functions to build **accurate**, **time-aware** and **reliable** DataWeave solutions!



4.10.3

DATE & TIME FUNCTIONS

 MuleSoft
from salesforce



Master Date & Time functions to write **accurate**,
time-aware and **reliable** DataWeave solutions!

QUICK REFERENCE TABLE

Function	Purpose
<code>now()</code>	Current date and time
<code>today()</code>	Current date
<code>format()</code>	Format date/time
<code>parse()</code>	Parse string into date/time
<code>add()</code>	Add duration/period
<code>subtract()</code>	Subtract duration/period
<code>withZone()</code>	Convert to different time zone
<code>between()</code>	Difference between dates
<code>isBefore()</code>	Check if date1 is before date2
<code>isAfter()</code>	Check if date1 is after date2
<code>atZone()</code>	Attach zone to local date/time

• SYNTAX CHEAT SHEET •

<code>now()</code>	<code>now()</code>
<code>today()</code>	<code>today()</code>
<code>format()</code>	<code>format(dateTime, pattern)</code>
<code>parse()</code>	<code>parse(string, pattern)</code>
<code>add()</code>	<code>add(dateTime, duration/period)</code>
<code>subtract()</code>	<code>subtract(dateTime, duration/period)</code>
<code>withZone()</code>	<code>withZone(dateTime, "Zone/Id")</code>
<code>between()</code>	<code>between(date1, date2, unit)</code>
<code>isBefore()</code>	<code>isBefore(date1, date2)</code>
<code>isAfter()</code>	<code>isAfter(date1, date2)</code>
<code>atZone()</code>	<code>atZone(localDateTime, "Zone/Id")</code>

REAL-WORLD SCENARIOS

 Log timestamps in UTC and convert at the edges.	Functions: <code>now()</code> , <code>withZone()</code>	Example: Store in UTC, display in user's time zone.
 Calculate SLAs and timeouts.	Functions: <code>add()</code> , <code>between()</code> , <code>today()</code>	Example: Check if SLA breached in hours.
 Handle user time zones for reports and notifications.	Functions: <code>withZone()</code> , <code>atZone()</code>	Example: Show dates in user's local time.
 Schedule jobs and reminders.	Functions: <code>add()</code> , <code>format()</code>	Example: Send reminder after 2 days.
 Validate data within specific date ranges.	Functions: <code>isBefore()</code> , <code>isAfter()</code> , <code>between()</code>	Example: Validate order date in range.

BEST PRACTICES

- Always use UTC for storing timestamps.
- Convert to user time zone only for display.
- Use ISO-8601 format for consistency.
- Handle daylight saving changes carefully.
- Validate date inputs before processing.

COMMON MISTAKES

- Parsing without specifying the correct pattern.
- Mixing up Duration and Period.
- Ignoring time zones and using server local time.
- Assuming `between()` returns fractional values.
- Not handling null or invalid date strings.

EXAMPLE COMBINATION

```
%dw 2.0
output application/json
var dt = "2024-05-20T14:35:22Z"
---
{
  localTime: withZone(dt, "Asia/Kolkata"), // convert zone
  formatted: format(dt, "yyyy-MM-dd HH:mm"), // format
  tomorrow: add(today(), |P1D|), // add 1 day
  diffInDays: between("2024-05-20", "2024-05-25", "days")
  // difference in days
}
```

KEY TAKEAWAYS

- ★ Date & Time functions make your scripts time-aware.
- ★ They help in calculations, comparisons and scheduling.
- ★ Use the right function for the right task.
- ★ Consistent handling of time zones ensures accuracy.
- ★ Combine multiple functions for powerful transformations.

SAMPLE OUTPUT

```
{
  "localTime": "2024-05-20T20:05:22+05:30",
  "formatted": "2024-05-20 14:35",
  "tomorrow": "2024-05-21",
  "diffInDays": 5
}
```

 Leverage Date & Time functions to build **accurate**,
time-aware and **reliable** DataWeave solutions! 

4.11

ALL DATAWEAVE FUNCTIONS



DataWeave provides a rich set of built-in functions to transform, manipulate, query and validate data easily. These functions are grouped by category.

1. STRING FUNCTIONS - upper() - lower() - trim() - sizeOf() - splitBy(text, separator) - joinBy(array, separator) - replace(text, old, new) - substring(text, from, to) - contains(text, substring) - startsWith(text, prefix) - endsWith(text, suffix) - matches(text, regex)	2. ARRAY FUNCTIONS - sizeOf(array) - isEmpty(array) - map(array, (item, index) -> ...) - filter(array, (item, index) -> ...) - reduce(array, (acc, item, index) -> ..., initialValue) - pluck(array, key) - distinctBy(array, criteria) - orderBy(array, (item) -> ...) - groupBy(array, (item) -> ...) - flatten(array) - slice(array, from, to) - take(array, n) - drop(array, n)	3. OBJECT FUNCTIONS - keys(object) - values(object) - pluck(object, key) - mapObject(object, (value, key) -> ...) - filterObject(object, (value, key) -> ...) ++ (obj1, obj2) - update(object, key, value) - removeKey(object, key) - isEmpty(object) - sizeOf(object)	4. DATE & TIME FUNCTIONS - now() - today() - format(dateTime, pattern) - parse(string, pattern) - add(dateTime, duration/period) - subtract(dateTime, duration/period) - withZone(dateTime, zone) - between(date1, date2, unit) - isBefore(date1, date2) - isAfter(date1, date2) - atZone(localDateTime, zone)
5. MATH FUNCTIONS - abs(number) - round(number) - floor(number) - ceil(number) - max(number1, number2, ...) - min(number1, number2, ...) - sum(array) - avg(array) - random()	6. LOGICAL FUNCTIONS - not(value) - and(value1, value2, ...) - or(value1, value2, ...) - default(value, defaultValue) - isEmpty(value) - isBlank(value) - isNull(value) - isNumber(value) - isString(value) - isBoolean(value)	7. TYPE & CONVERSION FUNCTIONS - asString(value) - asNumber(value) - asBoolean(value) - asArray(value) - asObject(value) - asDate(value, pattern) - asDateTime(value, pattern) - asLocalDate(value) - asLocalTime(value) - asPeriod(value) - asDuration(value)	8. NULL & DEFAULT FUNCTIONS - default(value, defaultValue) - whenNull(value, defaultValue) - whenEmpty(value, defaultValue) - isNull(value) - isEmpty(value) - isBlank(value)
9. QUERY FUNCTIONS - filter(array, condition) - find(array, condition) - findIndex(array, condition) - any(array, condition) - every(array, condition) - none(array, condition)	10. PAYLOAD NAVIGATION FUNCTIONS - payload - attributes - vars - correlationId - messageId - error - error.description - error.cause	11. XML & NAMESPACE FUNCTIONS - localName(name) - namespace(name) - prefix(name) - namespaceUri(name) - qName(prefix, localName)	12. MISCELLANEOUS FUNCTIONS - uuid() - now() \$\$ (current context) - write(value, format) - read(payload, format) - cache(key, ttl, factory) - do(expression)

KEY TAKEAWAYS



- ✓ DataWeave functions are **IMMUTABLE** and return new values.
- ✓ Choose the right function for the right task.
- ✓ Use functions to write **clean, readable and maintainable** scripts.
- ✓ Combine multiple functions for powerful transformations.
- ✓ Refer to official MuleSoft documentation for more details.

SYNTAX REMINDER

- functionName(arg1, arg2, ...) // most functions
- Use named parameters where available.
- Functions are case-sensitive.
- All functions return new results (do not modify originals).



Master DataWeave functions and build **efficient**,
clean and **scalable** MuleSoft integrations!



4.12.1 READ & WRITE FORMATS



DataWeave supports multiple data formats for **reading** input and **writing** output payloads.

Choosing the right format ensures seamless integration and optimal performance.

OVERVIEW

- ✓ Read formats parse incoming payloads into DataWeave data.
- ✓ Write formats serialize DataWeave data into the desired output.
- ✓ Formats can be configured using metadata or inline script directives.
- ✓ Custom properties provide fine-grained control over parsing and writing.

SUPPORTED FORMATS

Format	Description
<code>{ }</code> JSON	JavaScript Object Notation
<code>< ></code> XML	Extensible Markup Language
<code>CSV</code>	Comma-Separated Values
<code>JAVA</code>	Java Objects
<code>PROPERTIES</code>	Java Properties Files
<code>T</code> TEXT	Plain Text
<code>0110</code> BINARY	Binary Data (Base64)
<code>YAML</code>	YAML Ain't Markup Language

SYNTAX OVERVIEW

Read Format

```
%dw 2.0
input payload <formatName>
---
payload
```

Write Format

```
%dw 2.0
output <formatName>
---
<data>
```



- Format names are case-insensitive.
- Default read format is `application/java`.
- Default write format is `application/java`.

READ FORMATS (INPUT FORMATS)

1. JSON

Example Input

```
{
  "name": "John",
  "age": 30
}
```

DataWeave

```
%dw 2.0
input payload json
---
payload
```

Output

```
{
  "name": "John",
  "age": 30
}
```

2. XML

Example Input

```
<person>
  <name>John</name>
  <age>30</age>
</person>
```

DataWeave

```
%dw 2.0
input payload xml
---
payload
```

Output

```
{
  "person": {
    "name": "John",
    "age": "30"
  }
}
```

3. CSV

Example Input

```
name,age,city
John,30,NY
Jane,25,LA
```

DataWeave

```
%dw 2.0
input payload csv
---
payload
```

Output

```
[
  { "name": "John", "age": "30", "city": "NY" },
  { "name": "Jane", "age": "25", "city": "LA" }
]
```



Understanding input formats is the first step toward building **robust** DataWeave transformations.

4.12.2

READ & WRITE FORMATS



DataWeave supports multiple data formats for **reading** input payloads and **writing** output payloads. Choosing the right format ensures seamless integration and optimal performance.

READ FORMATS (INPUT FORMATS) – PART 2

4. PROPERTIES

Example Input

```
host=localhost
port=8080
env=dev
```

DataWeave

```
%dw 2.0
input payload properties
---
payload
```

Output (DataWeave Data)

```
{ "host": "localhost", "port": "8080", "env": "dev" }
```

5. BINARY

Example Input

```
(Binary file /
Base64 encoded
content)
```

DataWeave

```
%dw 2.0
input payload binary
---
payload
```

Output (DataWeave Data)

```
(Binary data as Base64 string)
"U29tZUJhc2U2NERhdGE="
```

6. TEXT

Example Input

```
Hello World!
Welcome to DataWeave.
```

DataWeave

```
%dw 2.0
input payload text
---
payload
```

Output (DataWeave Data)

```
"Hello World!\nWelcome to DataWeave."
```

7. YAML

Example Input

```
user:
  name: John
  age: 30
  city: New York
```

DataWeave

```
%dw 2.0
input payload yaml
---
payload
```

Output (DataWeave Data)

```
{ "user": { "name": "John", "age": "30",
"city": "New York" } }
```

WRITE FORMATS (OUTPUT FORMATS)

1. JSON

DataWeave

```
%dw 2.0
output application/json
---
{ "name": "John", "age": 30 }
```

2. XML

DataWeave

```
%dw 2.0
output application/xml
---
{ "name": "John", "age": 30 }
```

3. CSV

DataWeave

```
%dw 2.0
output application/csv
---
[["name": "John", "age": 30]]
```

4. PROPERTIES

DataWeave

```
%dw 2.0
output application/properties
---
{ "host": "localhost",
"port": "8080" }
```

5. TEXT

DataWeave

```
%dw 2.0
output application/text
---
"Hello World!"
```

6. BINARY

DataWeave

```
%dw 2.0
output application/binary
---
(Binary data / Base64)
```

7. YAML

DataWeave

```
%dw 2.0
output application/yaml
---
{ "user": { "name": "John" } }
```

8. JAVA

DataWeave

```
%dw 2.0
output application/java
---
{ "name": "John", "age": 30 }
```



KEY TAKEAWAYS

- Use **input payload <format>** to read data in a specific format.
- Use **output application/<format>** to write data in the desired format.
- Choose the right format for better performance and maintainability.
- Leverage custom properties for fine-grained control.

Remember:

Format names are case-insensitive.

4.12.3

READ & WRITE FORMATS



DataWeave supports multiple data formats for **reading** input payloads and **writing** output payloads.

Choosing the right format ensures seamless integration and optimal performance.

WRITE FORMATS (OUTPUT FORMATS) - PART 2

Format	Description	DataWeave Syntax (Output)	Example Output
 JAVA	Serializes data as Java Objects.	%dw 2.0 output application/java --- payload	(Java Object representation of the payload) e.g. Person {name=John, age=30}
 CSV	Serializes data into Comma-Separated Values.	%dw 2.0 output application/csv --- payload	name,age,city John,30,NY Jane,25,LA
 PROPERTIES	Serializes data as Java Properties (key=value pairs).	%dw 2.0 output application/properties --- payload	host=localhost port=8080 env=dev
 TEXT	Serializes data as plain text.	%dw 2.0 output application/text --- payload	Hello World! Welcome to DataWeave.
 BINARY	Serializes data as binary (Base64 encoded).	%dw 2.0 output application/binary --- payload	QmluYXJ5IGRhdGEgYXMg QmFzZTY0IQ== (Base64 encoded content)
 YAML	Serializes data as YAML Ain't Markup Language.	%dw 2.0 output application/yaml --- payload	user: name: John age: 30 city: New York

EXAMPLES SUMMARY (OUTPUT FORMATS)

1. JSON <pre>%dw 2.0 output application/json --- payload</pre> Example Output <pre>{ "name": "John", "age": 30, "city": "NY" }</pre>	2. XML <pre>%dw 2.0 output application/xml --- payload</pre> Example Output <pre><user> <name>John </name> <age>30 </age> <city>NY </city> </user></pre>	3. CSV <pre>%dw 2.0 output application/csv --- payload</pre> Example Output <pre>name,age,city John,30,NY Jane,25,LA</pre>	4. PROPERTIES <pre>%dw 2.0 output application/properties --- payload</pre> Example Output <pre>host=localhost port=8080 env=dev</pre>
5. TEXT <pre>%dw 2.0 output application/text --- payload</pre> Example Output <pre>Hello World! Welcome to DataWeave.</pre>	6. BINARY <pre>%dw 2.0 output application/binary --- payload</pre> Example Output <pre>QmluYXJ5IGRhdGEgYXMg QmFzZTY0IQ== (Base64 encoded content)</pre>	7. YAML <pre>%dw 2.0 output application/yaml --- payload</pre> Example Output <pre>user: name: John age: 30 city: New York</pre>	8. JAVA <pre>%dw 2.0 output application/java --- payload</pre> Example Output <pre>(Java Object representation of the payload) e.g. Person {name=John, age=30}</pre>

KEY TAKEAWAYS



- Use **output <format>** to write data in the desired format.
- Choose the right format for interoperability.
- Custom properties help control serialization behavior.
- Consistent formats ensure smooth system integrations.

REMEMBER



- Format names are case-insensitive.
- Default write format is **application/java**.
- Test with sample payloads for accurate results.
- Refer to MuleSoft DataWeave docs for advanced configuration.

4.13.1 XML TRANSFORMATIONS



XML transformations in DataWeave help you **convert**, **reshape** and **map** XML data into the required structures. They ensure clean, consistent and integration-ready payloads for real-world applications.

OVERVIEW

- ✓ DataWeave makes XML transformations simple and powerful.
- ✓ It helps you extract, restructure, rename and enrich XML nodes and values.
- ✓ Supports filtering, sorting, aggregation and complex mappings.
- ✓ Produces valid, clean, namespace-aware XML output.

COMMON XML TRANSFORMATION TASKS

 Element Mapping Map source elements to target elements	 Attribute Mapping Map and create attributes	 Filtering Filter elements or values
 Restructuring Change hierarchy or structure	 Namespaces Handle and declare namespaces	 Conditional Logic Apply conditions in mapping
 Aggregation Group and aggregate data	 Arrays & Repetition Handle arrays and repeat elements	

SYNTAX OVERVIEW

Root Element
`< rootName ... > ... </ rootName >`

Child Element
`< childName ... > ... </ childName >`

Attribute
`@ attributeName = "value"`

Namespace
`ns default "uri"`

Filter
`filter (condition)`

Map Array
`array map ((item, index) -> ...)`

Conditional
`if (condition) ... else ...`

Text (Value)
`"text value" or value`



Always output well-formed XML with a single root element. Declare namespaces for accuracy.

EXAMPLES - XML TRANSFORMATIONS (BASIC)

1. Simple Element Mapping

Input (XML)

```
<book>
  <id>123</id>
  <title>DataWeave Guide</title>
</book>
```

DataWeave

```
output application/xml
---
{
  bookId: payload.book.id,
  name: payload.book.title
}
```

Output (XML)

```
<library>
  <bookId>123</bookId>
  <name>DataWeave Guide</name>
</library>
```

2. Attribute Mapping

Input (XML)

```
<user id="U1001">
  <name>John</name>
</user>
```

DataWeave

```
output application/xml
---
{
  user: {
    @id: payload.user.@id,
    fullName: payload.user.name
  }
}
```

Output (XML)

```
<customer id="U1001">
  <fullName>John</fullName>
</customer>
```

3. Filtering Elements

Input (XML)

```
<orders>
  <order id="1" status="NEW" />
  <order id="2" status="CLOSED" />
  <order id="3" status="NEW" />
</orders>
```

DataWeave

```
output application/xml
---
orders: payload.orders filter
  ($ status == "NEW")
```

Output (XML)

```
<orders>
  <order id="1" status="NEW" />
  <order id="3" status="NEW" />
</orders>
```



These basic transformations form the foundation for advanced XML mappings in DataWeave.

1

4.13.2

XML TRANSFORMATIONS

Part 2
of 3

EXAMPLES - XML TRANSFORMATIONS (CONTINUED)

4. Restructuring

Input (XML)

```
<employee>
  <name>John</name>
  <dept>IT</dept>
  <role>Dev</role>
</employee>
```

DataWeave

```
output application/xml
---
{
  staff: {
    "person": payload.employee.name,
    "department": payload.employee.dept,
    "designation": payload.employee.role
  }
}
```

Output (XML)

```
<staff>
  <person>John</person>
  <department>IT</department>
  <designation>Dev</designation>
</staff>
```

5. Namespaces

Input (XML)

```
<ns:root xmlns:ns="http://ns.com">
  <ns:item>Values</ns:item>
</ns:root>
```

DataWeave

```
output application/xml
---
ns default "http://ns.com"
{
  items: {
    item: "Values"
  }
}
```

Output (XML)

```
<items xmlns="http://ns.com">
  <item>Values</item>
</items>
```

💡 Use namespaces to avoid name collisions and ensure XML validity.

6. Array Mapping

Input (XML)

```
<users>
  <user><name>Ac</name></user>
  <user><name>Bc</name></user>
</users>
```

DataWeave

```
output application/xml
---
{
  users:
    payload.users map ((u) => {
      userName: u.name
    })
}
```

Output (XML)

```
<users>
  <userName>Ac</userName>
  <userName>Bc</userName>
</users>
```

💡 Use map to transform each element in an array.

7. Conditional Mapping

Input (XML)

```
<user>
  <age>25</age>
  <status>Active</status>
</user>
```

DataWeave

```
output application/xml
---
<user>
  <age>{ payload.user.age }</age>
  <status>
    %if (payload.user.age >= 18)
      "Adult"
    %else
      "Minor"
    %end
  </status>
</user>
```

Output (XML)

```
<user>
  <age>25</age>
  <status>Adult</status>
</user>
```

💡 Use conditions to apply business rules in mapping.

ADVANCED TRANSFORMATION EXAMPLES

8. Aggregation

Input (XML)

```
<sales>
  <item amount="100" />
  <item amount="200" />
  <item amount="150" />
</sales>
```

DataWeave

```
output application/xml
---
<summary>
  <total>{ (payload.sales.item.@amount
    default {1} sum as Number )</total>
  <count>{ sizeOf(payload.sales.item) }</count>
</summary>
```

Output (XML)

```
<summary>
  <total>450</total>
  <count>3</count>
</summary>
```

💡 Use sum, count, sizeOf to perform aggregations on XML data.

9. Complex Transformation

Input (XML)

```
<orders>
  <order id="1" name="TV">
    <qty>2</qty> <price>300</price>
  </order>
  <order id="2" name="Table">
    <qty>1</qty> <price>120</price>
  </order>
</orders>
```

DataWeave

```
output application/xml
---
<invoice>
  <header>Invoice 001</header>
  <items>
    %for (o in payload.orders) {
      <product>{o.name}</product>
      <quantity>{o.qty}</quantity>
      <amount>{o.qty * o.price}</amount>
    }
  </items>
</invoice>
```

Output (XML)

```
<invoice>
  <header>Invoice 001</header>
  <items>
    <product>TV</product>
    <quantity>2</quantity>
    <amount>600</amount>
    <product>Table</product>
    <quantity>1</quantity>
    <amount>120</amount>
  </items>
</invoice>
```

Tips

- ✓ Use meaningful element names and attributes.
- ✓ Declare namespaces explicitly.
- ✓ Filter data early for better performance.
- ✓ Avoid deep nesting in XML whenever possible.
- ✓ Validate output XML using tools or schemas.
- ✓ Leverage DataWeave functions for mapping, filtering, aggregation and restructuring.



Remember!

Clean and well-structured XML leads to reliable integrations and easier maintenance.



Master XML transformations in DataWeave to build **clean, reliable** and **integration-ready** solutions!

2

4.13.3

XML TRANSFORMATIONS

Part 3
of 3

SYNTAX OVERVIEW

Item	DataWeave Syntax
Root Element	< rootName ... > ... </ rootName >
Child Element	< childName ... > ... </ childName >
Attribute	@ attributeName = "value"
Namespace	ns default "uri"
Filter	filter (condition)
Map Array	array map ((item, index) -> ...)
Conditional	if (condition) ... else ...
Text (Value)	"text value" or value



Always output **well formed XML** with a single root element.
Declare **namespaces** for accuracy and interoperability.

BEST PRACTICES

- ✓ Use clear and meaningful element and attribute names.
- ✓ Declare namespaces explicitly.
- ✓ Filter data early for better performance.
- ✓ Avoid deep nesting; keep XML structure clean.
- ✓ Validate output XML using tools or schemas.

KEY TAKEAWAYS

- ★ XML transformations convert source data into structured, usable XML output.
- ★ DataWeave provides powerful mapping, filtering, aggregation and restructuring.
- ★ Handle namespaces and attributes carefully for accurate integrations.
- ★ Clean and optimized transformations lead to reliable and performant solutions.

COMMON MISTAKES

✗ Forgetting the single root element.

✗ Not declaring namespaces (leading to invalid XML).

✗ Incorrect use of arrays and indexes.

✗ Missing attributes or wrong data types.

✗ Ignoring empty or null values.

QUICK REFERENCE

Element	Attribute	Namespace	Filter	Map Array	Conditional	Text Value
Syntax <name> ... </name>	Syntax @name = "value"	Syntax ns default "uri"	Syntax filter (condition)	Syntax array map ((item, index) -> ...)	Syntax if (condition) ... else ...	Syntax "text" or value
Example <book> <title>DataWeave Guide</title> </book>	Example <book id="123"> <title>Book</title> </book>	Example ns default "http:// example.com" <root> <child/> </root>	Example orders filter (\$ status == "NEW")	Example items map ((item) -> { <item>{item.name}</item> })	Example if (\$ age > 18) "Adult" else "Minor"	Example <name>{user.name}</name>

FULL WORKED EXAMPLE - INVENTORY REPORT

Input (XML)

```
<inventory>  
  <category name="Electronics">  
    <product id="P1" name="TV" price="300" qty="2" />  
    <product id="P2" name="Laptop" price="800" qty="1" />  
  </category>  
  <category name="Accessories">  
    <product id="P3" name="Mouse" price="20" qty="5" />  
  </category>  
</inventory>
```



Note
..product selects all product elements depth-first.

- `sizeOf()` counts total products.
- `reduce()` calculates total value.
- `map()` builds the products list.

DataWeave

```
output application/xml  
---  
<report>  
  <generatedDate>{now()}</generatedDate>  
  <summary>  
    <totalProducts>{payload..product.sizeOf()}</totalProducts>  
    <totalValue>{payload..product.map((p) ->  
      (p.@price as Number) * (p.@qty as Number))  
      reduce ((acc, item) -> acc + item, 0)}</totalValue>  
  </summary>  
  <products>  
    {payload..product.map((p) ->  
      <product>  
        <id>{p.@id}</id>  
        <name>{p.@name}</name>  
        <price>{p.@price}</price>  
        <quantity>{p.@qty}</quantity>  
        <value>{(p.@price as Number) * (p.@qty as Number)}</value>  
      }</product>  
    )}</products>  
</report>
```

Output (XML)

```
<report>  
  <generatedDate>2024-05-16T10:30:00Z</generatedDate>  
  <summary>  
    <totalProducts>3</totalProducts>  
    <totalValue>1460</totalValue>  
  </summary>  
  <products>  
    <product>  
      <id>P1</id>  
      <name>TV</name>  
      <price>300</price>  
      <quantity>2</quantity>  
      <value>600</value>  
    </product>  
    <product>  
      <id>P2</id>  
      <name>Laptop</name>  
      <price>800</price>  
      <quantity>1</quantity>  
      <value>800</value>  
    </product>  
    <product>  
      <id>P3</id>  
      <name>Mouse</name>  
      <price>20</price>  
      <quantity>5</quantity>  
      <value>100</value>  
    </product>  
  </products>  
</report>
```



With the right techniques, XML transformations in DataWeave become **clean**, **efficient** and **integration-ready**!

3

4.14.1 CSV TRANSFORMATIONS



CSV (Comma Separated Values) transformations in DataWeave help you **parse**, **write** and **manipulate** delimited data. They are simple, lightweight and ideal for flat-file integrations and data exchange.

OVERVIEW

- ✓ CSV is a plain text format where values are separated by delimiters (default comma).
- ✓ DataWeave reads CSV data into arrays of objects.
- ✓ You can transform, filter, sort and reshape CSV data easily.
- ✓ DataWeave can also write data back to CSV using custom options.

COMMON CSV TRANSFORMATION TASKS

	Read / Parse CSV
	Write CSV
	Filter Rows
	Sort Data
	Select / Rename Columns
	Add / Remove Columns
	Join / Lookup Data
	Aggregate Data
	Group / Pivot Data
	Custom Delimiters & Options

SYNTAX OVERVIEW

Purpose	DataWeave Syntax
Read CSV	<code>read(url, "application/csv")</code>
Write CSV	<code>write(payload, "application/csv")</code>
Separator	<code>separator : " "</code>
Header	<code>header : true</code>
Quote Character	<code>quote : "\""</code>
Escape Character	<code>escape : "\""</code>
Line Separator	<code>lineSeparator : "\n"</code>
Skip Header	<code>skipHeader : true</code>
Null Values	<code>nullValue : ""</code>
Encoding	UTF-8



Default settings:

- separator = ","
- header = true
- quote = "\""
- escape = "\""
- lineSeparator = "\n"

EXAMPLES - READ CSV (Parsing CSV to Data)

EXAMPLE INPUT (employees.csv)

```
id,name,dept,role,salary
1,John,IT,Developer,85000
2,Alice,HR,Analyst,60000
3,Bob,IT,Architect,105000
4,Carol,Finance,Manager,75000
```

1. Basic Read

```
%dw 2.0
output application/json
---
read(payload, "application/csv")
```

Output (JSON)

```
[
  {
    "id": "1",
    "name": "John",
    "dept": "IT",
    "role": "Developer",
    "salary": "85000"
  },
  ... more rows
]
```

2. Read Without Header

```
%dw 2.0
output application/json
---
read(payload, "application/csv",
{
  header : false
})
```

3. Custom Delimiter

```
%dw 2.0
output application/json
---
read(payload,
  "application/csv",
{
  separator : "|"
})
```

4. Skip Header Row

```
%dw 2.0
output application/json
---
read(payload,
  "application/csv",
{
  skipHeader : true
})
```



Reading CSV files is one of the most common DataWeave operations in MuleSoft integrations. Understanding **headers**, **delimiters**, and **parsing options** is essential for working with flat-file data.

4.14.2

CSV TRANSFORMATIONS

EXAMPLES - WRITE CSV (DATA To CSV)

1. Basic Write (Default)

DataWeave

```
%dw 2.0
output application/csv
---
payload
```

Input Data (Payload)

```
[
  {id: 1, name: "John",
   dept: "IT", role: "Dev",
   salary: 85000},
  {id: 2, name: "Alice",
   dept: "HR", role: "Analyst",
   salary: 60000}
]
```

Output (CSV)

```
id,name,dept,role,salary
1,John,IT,Dev,85000
2,Alice,HR,Analyst,60000
```

2. Write Without Header

DataWeave

```
%dw 2.0
output application/csv
header: false
---
payload
```

Output (CSV)

```
1,John,IT,Dev,85000
2,Alice,HR,Analyst,60000
```

💡 Use header: false to remove the header row in the output CSV.

3. Custom Delimiter (|)

DataWeave

```
%dw 2.0
output application/csv
separator: "|"
---
payload
```

Output (CSV)

```
id|name|dept|role|salary
1|John|IT|Dev|85000
2|Alice|HR|Analyst|60000
```

💡 Change separator to any character you need (e.g. |, ;, ~).

4. Custom Quote & Escape

DataWeave

```
%dw 2.0
output application/csv
quote: ""
escape: "\\ "
---
payload
```

Output (CSV)

```
'id','name','dept','role','salary'
'1','John','IT','Dev','85000'
'2','Alice','HR','Analyst','60000'
```

💡 Customize quote and escape characters for special requirements.

COMMON TRANSFORMATIONS WITH CSV DATA

1. Filter Rows

DataWeave

```
%dw 2.0
output application/csv
---
payload filter ($.dept == "IT")
```

Output (CSV)

```
id,name,dept,role,salary
1,John,IT,Dev,85000
3,Bob,IT,Architect,105000
```

2. Sort By Salary (Desc)

DataWeave

```
%dw 2.0
output application/csv
---
payload orderBy (-$.salary)
```

Output (CSV)

```
id,name,dept,role,salary
3,Bob,IT,Architect,105000
1,John,IT,Dev,85000
4,Carol,Finance,Mgr,75000
2,Alice,HR,Analyst,60000
```

3. Select / Rename Columns

DataWeave

```
%dw 2.0
output application/csv
---
payload map {
  id: $.id,
  employee: $.name,
  department: $.dept,
  compensation: $.salary
}
```

Output (CSV)

```
id,employee,department,compensation
1,John,IT,85000
2,Alice,HR,60000
3,Bob,IT,105000
4,Carol,Finance,75000
```

4. Add / Remove Column

DataWeave

```
%dw 2.0
output application/csv
---
payload map (item) ->
  item ++ {
    bonus: item.salary * 0.1
  }
```

Output (CSV)

```
id,name,dept,role,salary,bonus
1,John,IT,Dev,85000,8500
2,Alice,HR,Analyst,60000,6000
3,Bob,IT,Architect,105000,10500
4,Carol,Finance,Mgr,75000,7500
```

5. Group / Aggregate

DataWeave

```
%dw 2.0
output application/csv
---
payload groupBy $.dept
map (dept, items) -> {
  department: dept,
  totalSalary: (items sum $.salary)
}
```

Output (CSV)

```
department,totalSalary
IT,190000
HR,60000
Finance,75000
```

CSV FORMAT OPTIONS

Option	Description	Example
separator	Field delimiter	"," or " "
header	Print header row	true / false
quote	Quote character	" or "
escape	Escape character	"\" or "\\ "
lineSeparator	Line ending	"\n" or "\r\n"
nullValue	String for nulls	"" or "NULL"
skipHeader	Skip first row	true / false
encoding	Character encoding	UTF-8

BEST PRACTICES

- ✓ Always specify header: true unless not needed.
- ✓ Use explicit data types when doing calculations.
- ✓ Handle nulls and empty values using nullValue.
- ✓ Validate delimiters, quotes and escape settings.
- ✓ Use streaming (readLH) for large CSV files.

COMMON MISTAKES

- ✗ Forgetting header: true leading to wrong field names.
- ✗ Using wrong delimiter causing parsing errors.
- ✗ Not handling quotes and escapes in text fields.
- ✗ Assuming all values are numbers or non-null.
- ✗ Writing large data without streaming or batching.

KEY TAKEAWAYS

- ★ CSV transformations in DataWeave are simple and efficient.
- ★ Read CSV into arrays of objects, transform and write back easily.
- ★ Flexible options help handle different CSV formats.
- ★ Good practices ensure clean, reliable and maintainable CSV integrations.



Transform flat files into valuable data with
DataWeave CSV transformations!



4.14.3 CSV TRANSFORMATIONS

— ADVANCED TRANSFORMATIONS & REAL-WORLD EXAMPLES —

1. Advanced Transformations with CSV Data

A) Filter by Salary (> 70000)

DataWeave

```
%dw 2.0
output application/csv
---
payload filter
($ salary as Number > 70000)
```

Output (CSV)

```
id,name,dept,role,salary
1,John,IT,Developer,85000
3,Bob,IT,Architect,105000
```

B) Sort by Salary (Descending)

DataWeave

```
%dw 2.0
output application/csv
---
payload orderBy
(-$.salary as Number)
```

Output (CSV)

```
id,name,dept,role,salary
3,Bob,IT,Architect,105000
1,John,IT,Developer,85000
4,Carol,Finance,Manager,75000
2,Alice,HR,Analyst,60000
```

C) Select / Rename Columns

DataWeave

```
%dw 2.0
output application/csv
---
payload map {
  ID : $.id,
  Employee : $.name,
  Department : $.dept,
  Compensation : $.salary
}
```

Output (CSV)

```
ID,Employee,Department,Compensation
1,John,IT,85000
2,Alice,HR,60000
3,Bob,IT,105000
4,Carol,Finance,75000
```

D) Add / Remove Column

DataWeave

```
%dw 2.0
output application/csv
---
payload map (item) -> {
  item ++ { Bonus: item.salary * 0.1 }
}
```

Output (CSV)

```
id,name,dept,role,salary,Bonus
1,John,IT,Developer,85000,8500
2,Alice,HR,Analyst,60000,6000
3,Bob,IT,Architect,105000,10500
4,Carol,Finance,Manager,75000,7500
```

E) Group / Aggregate by Department (Total Salary)

DataWeave

```
%dw 2.0
output application/csv
---
payload groupBy $.dept
map (dept, items) -> {
  Department: dept,
  TotalSalary: (items sum $.salary)
}
```

Output (CSV)

```
Department,TotalSalary
IT,190000
HR,60000
Finance,75000
```

2. CSV FORMAT OPTIONS (Reference)

Option	Description	Example
separator	Field delimiter	","; " "; ";"
header	Print header row	true / false
quote	Quote character	" "
escape	Escape character	"\"
lineSeparator	Line ending	"\n" or "\r\n"
nullValue	String for nulls	" " or "NULL"
skipHeader	Skip first row	true / false
encoding	Character encoding	UTF-8

3. REAL-WORLD SCENARIOS

- ★ Import and transform flat files from legacy systems.
- ★ Generate CSV reports for business users.
- ★ Exchange data with partners using CSV files.
- ★ Convert CSV data to JSON/XML for API processing.
- ★ Cleanse, enrich and reshape CSV data for analytics.
- ★ Load CSV data into databases or data lakes.

4. BEST PRACTICES

- ✓ Always specify header: true unless not needed.
- ✓ Use explicit data types when doing calculations.
- ✓ Handle nulls and empty values using nullValue.
- ✓ Validate delimiters, quotes and escape settings.
- ✓ Use streaming (readUrl) for large CSV files.
- ✓ Keep transformations simple and test with samples.

5. COMMON MISTAKES

- ✗ Forgetting header: true leading to wrong field names.
- ✗ Using wrong delimiter causing parsing errors.
- ✗ Not handling quotes and escapes in text fields.
- ✗ Assuming all values are numbers or non-null.
- ✗ Writing large data without streaming or batching.

6. QUICK REFERENCE

Task	DataWeave Example
Read CSV	read(url, "application/csv")
Write CSV	write(payload, "application/csv")
Filter Rows	payload filter (condition)
Sort Rows	payload orderBy (\$.field)
Add Column	payload map (item) -> item ++ { ... }
Group / Agg.	payload groupBy \$.field
Select Columns	payload map { field1: \$.a, ... }

7. KEY TAKEAWAYS

- ★ CSV transformations in DataWeave are simple, powerful and efficient.
- ★ Use read() to parse CSV into arrays and write() to export back to CSV.
- ★ Advanced operations like filter, sort, group, aggregate and column manipulations are easy.
- ★ Proper options and best practices ensure clean, reliable and scalable CSV integrations.



Master CSV transformations in DataWeave –
Transform flat files into valuable data!



4.15 ADVANCED DATAWEAVE



Advanced DataWeave helps you solve complex integration challenges with **reusable functions**, **user-defined types**, **modules**, **higher-order functions** and **advanced expression**. Write concise, maintainable and high-performance transformations.

OVERVIEW

- ✓ Go beyond basic mapping with functions, modules and types.
- ✓ Build reusable, maintainable and testable transformations.
- ✓ Leverage higher-order functions for dynamic data handling.
- ✓ Handle complex data structures, logic and constraints.
- ✓ Improve performance and consistency using modular code and reusability.

ADVANCED CAPABILITIES

User Defined Functions	User Modules	User Defined Types	Lambda Expressions
Higher-Order Functions	Recursion	Pattern Matching	Conditional Expressions
Namespaces & Imports	Metadata Access	Dynamic Keys	Error Handling & Defaults

SYNTAX OVERVIEW

Function	Syntax (Example)
Import	<code>import * from dw::core::Strings*</code>
Define function	<code>fun name(p1, p2, ...) = ...</code>
Lambda	<code>(item) -> item + 1</code>
User defined type	<code>type Name = { field: String, age: Number }</code>
Array of objects	<code>[{ name: "John", age: 30 }]</code>
Map (Object)	<code>{ key1: "value1", key2: "value2" }</code>
Default (null coalesce)	<code>payload.name default "Unknown"</code>
If / Else	<code>if (condition) "Yes" else "No"</code>
Match (pattern)	<code>match (payload) { case "A" -> 1 }</code>
Try / Catch	<code>try { ... } catch (e) { ... }</code>

EXAMPLES - ADVANCED PATTERNS

- ① **User Defined Function**
`fun toTitle(str) = str upper`
- ② **Module Import & Use**
`import * from modules::strUtils`
- ③ **Lambda & Map**
`[1, 2, 3] map ((n) -> n * 2)`
- ④ **Higher-Order Function**
`payload filter ((n) -> n > 10)`
- ⑤ **Pattern Matching**
`match payload type
case "user" -> "USER DATA"
case "admin" -> "ADMIN DATA"
else -> "UNKNOWN"`
- ⑥ **Try / Catch**
`try { payload map (n) -> n.id }
catch (e) { [] }`



Write once, reuse everywhere.
Modularize and simplify complex transformations.

BUILT-IN FUNCTIONS (Advanced)

Function	Description	Example
<code>sizeOf()</code>	Length of array/string	<code>sizeOf([1, 2, 3]) -> 3</code>
<code>flatten()</code>	Flatten nested arrays	<code>flatten([1, [2]]) -> [1, 2]</code>
<code>distinctBy()</code>	Unique by key	<code>distinctBy(\$, \$.id)</code>
<code>groupBy()</code>	Group by key	<code>groupBy(\$, \$.cat)</code>
<code>orderBy()</code>	Sort by key	<code>orderBy(\$, \$.age)</code>
<code>filter()</code>	Filter items	<code>arr filter (\$.age > 18)</code>
<code>pluck()</code>	Extract field	<code>arr pluck \$.name</code>

COMMON MISTAKES

- ✗ Parsing without specifying the correct pattern.
- ✗ Mixing up brackets and parentheses.
- ✗ Ignoring time zones and using server local time.
- ✗ Assuming all values are numbers or non-null.
- ✗ Not handling null or invalid data strings.

KEY TAKEAWAYS

- ★ Advanced DataWeave enables powerful and reusable solutions.
- ★ Use functions, types and modules to write clean, maintainable code.
- ★ Leverage higher-order functions and pattern matching for flexible transformations.
- ★ Modular code leads to consistent and high-performance integrations.



Master Advanced DataWeave – Build **reusable**, **reliable** and **high-performance** integrations!



4.16 ERROR HANDLING IN DATAWEAVE

DataWeave provides powerful constructs to **handle errors gracefully**, prevent runtime failures and return meaningful **defaults** or **custom messages**, ensuring resilient and robust integrations.

OVERVIEW

- ✓ Runtime errors happen when expressions fail due to invalid inputs, missing data, etc.
- ✓ Error handling improves flow reliability and ensures predictable outputs.
- ✓ Use built-in functions, try/catch, defaults and custom logic to handle errors.
- ✓ Keep transformations safe, clean and user-friendly.

TYPES OF RUNTIME ERRORS

	Null Access Occurs when a field or function on null is accessed. Example: <code>payload.name</code>
	Type Mismatch Occurs when performed an incompatible data type. Example: <code>"abc" + 10</code>
	Invalid Cast Cannot cast a value to the target type. Example: <code>"abc" as Number</code>
	Invalid Function Argument Passing incorrect arguments to functions. Example: <code>sizeof(null)</code>
	Other Runtime Errors Index out of bounds, invalid regex, division by zero, etc.

SYNTAX OVERVIEW

Construct	Syntax / Example
Try / Catch	<code>try { <expression> } catch (e) { <fallback> }</code>
Try / Catch (with variable)	<code>try { <expression> } catch (e) { <fallback> }</code>
Default operator (when null)	<code><expression> default <value></code>
Default operator (when empty)	<code><expression> default <value> when "N/A"</code>
Exists operator	<code>is Number(payload.age) is String(payload.name)</code>
Type check	<code>type Name</code>

 **TIP**

- Prefer safe navigation (`?`) and defaults operator to avoid unnecessary errors.
- Use try/catch for complex logic that may fail.

EXAMPLES – TRY / CATCH

1. Basic Try / Catch <pre>try { "Hello " ++ "John" } catch (e) { "Unknown" }</pre> Output "Hello John"	2. Using Error Variable <pre>try { payload.age as Number } catch (e) { "Error: " ++ e.message }</pre> Output "Error: Cannot cast 'abc' to Number"	3. Nested Try / Catch <pre>try { try { payload.date when is Number(payload.date) } catch (e) { 0 } } catch (e) { -1 }</pre> Output 0 or -1 or number	4. Handling Function Errors <pre>try { sizeof(payload.items) } catch (e) { 0 } try { 100 / sizeof(payload.items) } catch (e) { 0 }</pre> Output 100 or 0 or use of array	5. Safe Navigation – Try / Catch <pre>try { payload.order.customer?.name } catch (e) { "N/A" }</pre> Output "N/A" when anything fails
--	---	--	--	---

DEFAULTING FUNCTIONS

1. Default when Null <code>payload.name default "N/A"</code> Output: "N/A"	3. Exists Operator <code>is Number(payload.age)</code> Output: true or false
2. Default when Empty <code>payload.items default []</code> Output: [] (empty array)	4. Type Check <code>is Number(payload.age)</code> Output: true or false

CUSTOM ERROR MESSAGES

```
try {
  payload.order.total as Number
} catch (e) {
  error "VAL:01" : "Invalid total.  
Please provide a valid number."
}
```

Result
"Invalid total: Please provide a valid number."

COMMON MISTAKES

- ✗ Not handling errors leads to flow failures.
- ✗ Ignoring null and runtime errors.
- ✗ Using try/catch for simple validations.
- ✗ Returning wrong data types in fallbacks.
- ✗ Returning errors instead of using values.

BEST PRACTICES

- ✓ Prefer safe navigation (`?`) and default operator.
- ✓ Use try/catch only for complex operations or expressions.
- ✓ Handle nulls and exceptions explicitly.
- ✓ Log or propagate error messages when needed.

KEY TAKEAWAYS

- ★ Advanced DataWeave enables powerful, resilient integrations.
- ★ Use functions, modules, types and higher-order functions.
- ★ Write clean, efficient and failure-ready transformations.
- ★ Handle errors gracefully for robust and user-friendly flows.



Handle errors gracefully. Build resilient, reliable and user-friendly integrations with DataWeave!



5.1 MULE FLOW



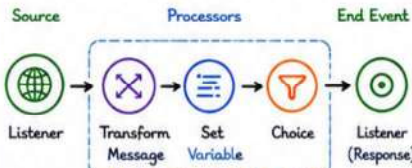
A Mule Flow is the **fundamental building block** of any Mule application. It defines a **sequence of components** that **process incoming events** to produce the **desired output**.

WHAT IS A FLOW?

- A flow is a message processing pipeline that starts with an event source and ends with an event.
- A flow contains a sequence of processors that act on the event.
- Each flow must have exactly one source.
- A flow can have zero or more message processors.
- A flow can have an optional error handling strategy.

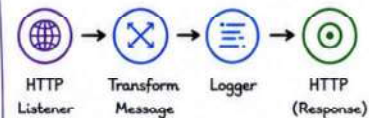
FLOW STRUCTURE

A typical Mule Flow consists of the following elements:



Source : Entry point of the flow. It receives an event.
Processors : Components that process/transform/filter/route the event.
End Event : The event is sent back to the caller or another flow.
Error Handling : Defines how errors in the flow will be handled.

SIMPLE FLOW EXAMPLE

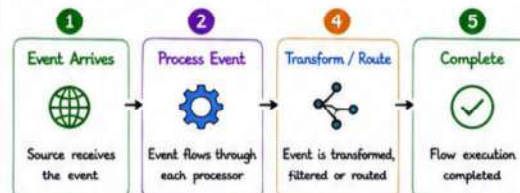


TIP

- Prefer safe navigation (?) and defaults operator to avoid unnecessary errors.
- Use try/catch for complex logic that may fail.

EVENT PROCESSING IN A FLOW

1. An event is received by the source (e.g., HTTP Listener).
2. The event passes through each processor in sequence.
3. Each processor can transform, filter, enrich, or route the event.
4. The event is returned by the end event (e.g., HTTP Response).
5. If any error occurs, the error handler is triggered.



FLOW EXAMPLE - REAL WORLD



Key Points:

- ✓ Flow starts with HTTP Listener.
- ✓ Order is validated.
- ✓ Transformation is done using DataWeave.
- ✓ Request is sent to external system.
- ✓ Response is logged and returned to caller.

ERROR HANDLING IN FLOW

A flow can have an error handler to manage exceptions. You can define specific or global error handlers.



Common Errors Handled

- Null or missing data
- Invalid data type
- Invalid cast
- External system failure
- Timeout or connection error

BEST PRACTICES

- ✓ Keep flows short and focused on a single responsibility.
- ✓ Use meaningful names for flows and components.
- ✓ Use Logger components for monitoring and debugging.
- ✓ Handle errors gracefully using error handling strategies.
- ✓ Externalize configurations and endpoints.
- ✓ Reuse logic using Sub Flows or Flow References.

COMMON USE CASES

- ☁ Expose APIs
- 🗄 Data Transformation
- ⚙ System Integration
- 🔗 Message Routing
- 📄 File / Data Processing

INTERVIEW QUESTIONS

1. What is a Mule Flow?
2. Can a flow have multiple sources?
3. How does event processing work in a flow?
4. What is the difference between a flow and a sub flow?
5. How do you handle errors in a flow?
6. Can a flow return a response? If yes, how?

QUICK REFERENCE

Flow	A sequence of components that processes an event.
Source	Entry point of the flow.
Processor	Components that perform actions on the event.
End Event	Returns the event to the caller.
Error Handler	Handles exceptions in the flow.



Flows are the heart of Mule applications -
Keep them **simple**, **reusable** and **reliable**!



5.2 SUB FLOW

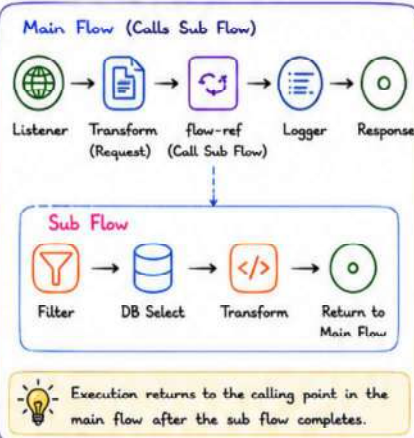


A Sub Flow is a **reusable** flow that can be called from **any flow** in the same application. It promotes **reusability**, **modularity** and **maintainability** of Mule applications.

WHAT IS A SUB FLOW?

- ✓ A Sub Flow is a flow defined with the **sub-flow** tag.
- ✓ It can be invoked using the **flow-ref** component.
- ✓ It runs in the same thread and shares the same event.
- ✓ It cannot be called from another application.
- ✓ It does not have a source or listener.

HOW SUB FLOW WORKS



SUB FLOW EXAMPLE

- 1) Main Flow**
 - HTTP Listener receives request.
 - Calls Sub Flow using flow-ref.
 - Logs response and returns to client.
- 2) Sub Flow Definition**
 - Validates order details.
 - Performs database operation.
 - Transforms data.
 - Returns control to main flow.

FLOW INVOCATION USING flow-ref

Attribute	Description
name	Name of the Sub Flow to invoke
doc:name	Display name in Studio
variables	Optional. Pass variables to sub flow

VARIABLES IN SUB FLOW

- ✓ Sub Flow can access and modify flow variables.
- ✓ Variables passed using "variables" are available inside the sub flow.
- ✓ Changes made to variables in sub flow are visible to the calling flow.

SUB FLOW vs PRIVATE FLOW

Aspect	Sub Flow	Private Flow
Visibility	Can be called from any flow in the same app	Can be called only within the same flow
Called using	flow-ref	flow-ref
Use case	Reuse logic across multiple flows	Break a large flow into smaller parts
Scope	Application level	Flow level (private)
Best for	Common business logic reuse	Improving readability of a single flow

REAL WORLD EXAMPLE



SUB FLOW BEST PRACTICES

- ✓ Keep sub flows small, focused and single-purpose.
- ✓ Use meaningful names and documentation.
- ✓ Avoid complex transformations inside sub flows.
- ✓ Use variables to pass data clearly.
- ✓ Handle errors inside sub flows and return meaningful errors.
- ✓ Test sub flows independently.

COMMON USE CASES

- ✓ Reusable business logic (e.g., validation, enrichment)
- ✓ Database operations
- ✓ Common transformations
- ✓ Audit logging
- ✓ Security checks
- ✓ External service calls

ERROR HANDLING IN SUB FLOW

- A sub flow can have an error handler to manage exceptions. Define specific or global error handlers.
- Common errors handled:
- Null or missing data
 - Invalid data type
 - Invalid cast
 - External system failure
 - Timeout or connection error

INTERVIEW QUESTIONS

1. What is a Sub Flow in Mule?
2. How do you invoke a Sub Flow?
3. Can a Sub Flow be called from another application?
4. How are variables shared between main flow and sub flow?
5. What is the difference between Sub Flow and Private Flow?
6. Can a Sub Flow have a listener?
7. What are the best practices for using Sub Flows?

QUICK REFERENCE

Concept	Description
Sub Flow	A reusable flow called from any flow in the same application.
Invoke Sub Flow	Using flow-ref component.
Variables	Use "variables" to pass data to sub flow.
Return	Control returns to main flow after sub flow completes.



Sub Flows make your integrations modular, reusable and easy to maintain!



5.3 PRIVATE FLOW



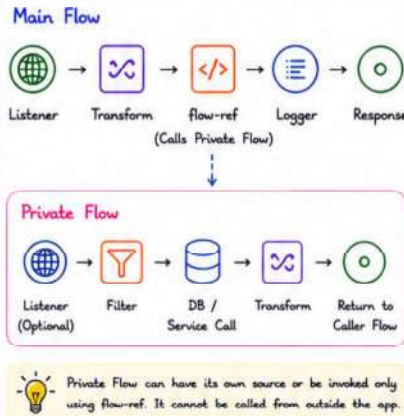
A Private Flow is a flow that is visible and callable only within the same Mule application.

It is similar to a Sub Flow but can contain its own sources or listeners.

WHAT IS A PRIVATE FLOW?

- ✓ A flow that is private to the current application.
- ✓ Can be called only from other flows in the same app.
- ✓ Can have a source (e.g., Listener).
- ✓ Not accessible from other applications.
- ✓ Ideal for breaking a large flow into smaller, reusable and independent triggers if needed.

PRIVATE FLOW CONCEPT



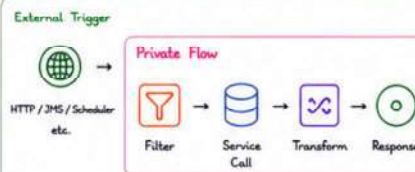
PRIVATE FLOW EXAMPLE

- 1) Main Flow → calls Private Flow
 - Receives order request.
 - Calls Private Flow using flow-ref.
 - Logs response and returns to client.
- 2) Private Flow → with Listener (optional)
 - Processes order: validates, updates DB, sends email.
 - Returns result to main flow.
 - Not accessible from outside the application.

SUB FLOW vs PRIVATE FLOW

Aspect	Sub Flow	Private Flow
Visibility	Can be called from any flow in the app	Can be called only within the same app
Called using	flow-ref	flow-ref
Use case	Reuse logic across multiple flows	Break a large flow into smaller parts
Source	No source or listener	Can have source (Listener optional)
Scope	Application level	Application level (Private)
Best for	Common business logic reuse	Independent triggers, modularity & flexibility

PRIVATE FLOW WITH LISTENER (INDEPENDENT TRIGGER)



Notes

- A Private Flow with a listener behaves like any other flow when triggered.
- It is still private to the application and cannot be invoked from other apps.

WHEN TO USE PRIVATE FLOW?

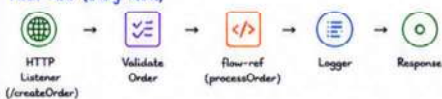
- ✓ When a flow needs to be invoked both internally and externally.
- ✓ When you want to encapsulate a complete process behind a listener.
- ✓ When you need to isolate business logic.
- ✓ When modularizing an application with reusable independent triggers.

Tip: Prefer Private Flow over Sub Flow when independent invoking is needed.

EXAMPLE: ORDER PROCESSING APPLICATION

Scenario: Create an order and send confirmation email.

Main Flow (Entry Point)



Private Flow (Reusable + Independent)



SUB FLOW BEST PRACTICES

- ✓ Keep private flows small, focused and single-purpose.
- ✓ Use meaningful names and documentation.
- ✓ Avoid complex transformations inside private flows.
- ✓ Use variables to pass data clearly.
- ✓ Handle errors inside private flows and return meaningful results.
- ✓ Test private flows independently.

COMMON USE CASES

- ✓ Reusable business logic (validation, enrichment)
- ✓ Database operations
- ✓ Common transformations
- ✓ Audit logging
- ✓ Security checks
- ✓ External service calls

ERROR HANDLING IN PRIVATE FLOW

Handle errors within the private flow and propagate custom error if needed.



Handle:

- Null or missing data
- Invalid data type
- Invalid cast
- External system failure
- Timeout or connection error

INTERVIEW QUESTIONS

1. What is a Private Flow in Mule?
2. How do you invoke a Private Flow?
3. Can a Private Flow be called from another application?
4. Can a Private Flow have a source or listener?
5. How are variables shared between main flow and private flow?
6. What is the difference between Sub Flow and Private Flow?
7. What are the best practices for using Private Flows?

QUICK REFERENCE

Concept	Description
Private Flow	A reusable flow visible and callable only within the same application.
Invoke Private Flow	Using flow-ref component.
Source / Listener	Optional. Makes the private flow independently triggerable.
Variables	Use "variables" to pass data between flows.
Return	Automatic return to caller flow after completion.



Private Flows give you flexibility and modularity with independent triggers!



5.4 FLOW REFERENCE

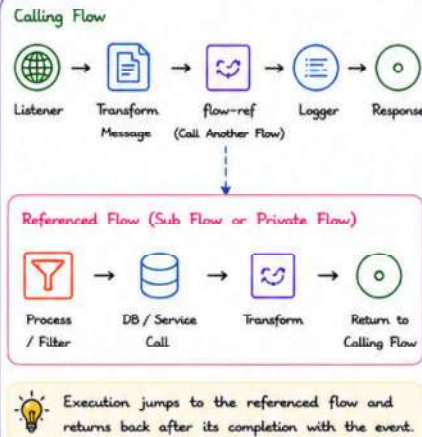


The **flow-ref** component is used to invoke a **Sub Flow** or a **Private Flow**. It promotes **reusability**, **modularity** and **maintainability** in Mule applications.

WHAT IS FLOW REFERENCE?

- ✓ flow-ref is a component used to call a Sub Flow or Private Flow.
- ✓ It passes the current Mule Event to the referenced flow.
- ✓ When the referenced flow completes, control returns to the calling flow.
- ✓ It helps in reusing business logic and keeping flows clean and manageable.

HOW FLOW-REF WORKS



SYNTAX

Basic Syntax

```
<flow-ref name="flow-name" />
```

With Variables

```
<flow-ref name="flow-name">
  <variables>
    <variable key="var1" value="{payload.id}" />
    <variable key="var2" value="{vars.customer}" />
  </variables>
</flow-ref>
```

Attribute	Description
name	Name of the Sub Flow or Private Flow to invoke.
doc:name	Display name in Anypoint Studio.
variables	Optional variables passed to the referenced flow.

PASSING VARIABLES TO FLOW-REF

Example

- Pass variables like orderId, payload, or sourceSystem using **<variables>**.
- Access in the referenced flow using **vars.orderId**, **vars.sourceSystem**.

RETURNING DATA FROM REFERENCED FLOW

- ✓ The referenced flow can set payload, variables or attributes.
- ✓ When it completes, the event (with updated payload/variables) returns to the calling flow.
- ✓ No additional configuration is required to return data.

FLOW-REF WITH SUB FLOW vs PRIVATE FLOW

Aspect	Sub Flow	Private Flow
Visibility	Visible only in current app	Visible within the same app
Called using	flow-ref	flow-ref
Source / Listener	Not allowed	Allowed (optional)
Reuse Scope	Across multiple flows	Within the same flow
Best For	Common reusable logic	Break large flows into small parts

REAL WORLD EXAMPLE

Scenario: Place Order and Send Confirmation (using Sub Flow)
Main Flow (Entry Point)



Sub Flow: place-order-subflow



ERROR HANDLING WITH FLOW-REF

Errors in the referenced flow propagate back to the calling flow.
 Handle using Try scope or Error Handler.

Common Errors:

- Flow not found
- Invalid variables
- Required variables missing
- Data transformation errors
- Business logic exceptions



BEST PRACTICES

- ✓ Use flow-ref to build reusable flows.
- ✓ Keep sub flows small and focused.
- ✓ Use meaningful names.
- ✓ Pass only necessary variables.
- ✓ Handle errors inside referenced flows and return meaningful messages.
- ✓ Avoid deep nesting of flow-refs.
- ✓ Prefer Private Flow for internal modularization.

WHEN TO USE PRIVATE FLOW?

- ✓ When a flow needs to be invoked both internally and externally.
- ✓ When you want to encapsulate a complete process behind a listener.
- ✓ When you need to isolate business logic.
- ✓ When modularizing an application with independent triggers.

Tip: Prefer **Private Flow** over Sub Flow when independent invocation is needed.

COMMON USE CASES

- Reusable business logic (validation, enrichment)
- Database operations
- Common transformations
- Audit logging
- Security / Authorisation checks
- External service calls

QUICK REFERENCE

Concept	Description
flow-ref	Invokes a Sub Flow or Private Flow.
Sub Flow	Reusable flow with no source or listener.
Private Flow	Flow with optional source/listener; callable internally and externally.
Variables	Pass data between calling flow and referenced flow.
Return	Control returns to the calling flow after completion.



Use **flow-ref** to build **reusable**, **modular** and **easy to maintain** Mule applications!



5.5 LOGGER COMPONENT

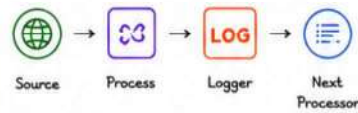


Logger component is used to log messages at various levels to **monitor** or **debug** Mule applications. It helps in **troubleshooting** and **tracking flow** execution, errors and data.

WHAT IS LOGGER COMPONENT?

- ✓ Logger writes messages to the specified log.
- ✓ It helps in monitoring, debugging and auditing.
- ✓ It does not change the message payload.
- ✓ It can log static text or dynamic data using DataWeave expressions.
- ✓ Available in all Mule editions.

HOW LOGGER WORKS



Logger listens to the flow and writes the provided message at the specified level without affecting the event.

BASIC SYNTAX

```
<logger level="LEVEL" message="MESSAGE" />
```

Where,

level : Logging level (INFO, DEBUG, ERROR, WARN, TRACE)

message : Message to be logged (String or DataWeave expression)

Attribute	Description
level	Specifies the log level.
message	The message to log. Can be static text or DataWeave expression.

LOG LEVELS

Level	Purpose
TRACE	Detailed diagnostic information
DEBUG	Detailed debug information
INFO	General operational messages
WARN	Warning messages
ERROR	Error messages

LOGGER IN A FLOW



Example

```
<logger level="INFO" message="Order received successfully!" />
```

LOGGER EXAMPLES

1. Static Message

```
<logger level="INFO" message="Order received successfully!" />
```
2. Dynamic Message Using DataWeave

```
<logger level="INFO" message="Order ID is " ++ payload.id ++ ", Customer: " ++ payload.customer />
```
3. Log Error Details

```
<logger level="ERROR" message="Error: " ++ vars.errorMessage ++ ", Action: " ++ vars.action />
```

LOG FILE PATTERN

Mule applications log to files in the following location:

```
${MULE_HOME}/logs/<app-name>.log
```

Sample Log Output

```
2024-05-20 10:15:20,123 INFO [create-order-flow] Logger - Order created successfully
2024-05-20 10:15:20,456 DEBUG [create-order-flow] Logger - Payload after transformation
2024-05-20 10:15:20,789 ERROR [create-order-flow] Logger - Failed to create order: Duplicate key
```

BEST PRACTICES

- ✓ Use meaningful log messages.
- ✓ Use appropriate log levels.
- ✓ Avoid logging sensitive information.
- ✓ Use DataWeave to log only required information.
- ✓ Remove or lower log levels in production.
- ✓ Use correlation IDs to trace requests.
- ✓ Keep logger messages short and clear.

COMMON USE CASES

- 🔍 Debugging and troubleshooting
- 📊 Monitoring flow execution
- 📄 Auditing business transactions
- 🔗 Tracking request/response
- 🕒 Performance analysis
- ⚠️ Error tracking and analysis

CONFIGURE LOGGER LEVEL (log4j2.xml)

You can control which levels are written to logs.

```
<configuration status="WARN">
  <appenders>
    <console name="Console" target="SYSTEM_OUT">
      <PatternLayout pattern="%d [%t] %-5level: %logger [%M] - %msg%n" />
    </console>
  </appenders>
  <loggers>
    <root level="INFO" additivity="false">
      <appenderRef ref="Console" />
    </root>
  </loggers>
</configuration>
```



Only messages at the configured level and above will be written to the logs.

CONDITIONAL LOGGING

Log only when condition is true.

```
<choice>
  <when expression="!${payload.isError}">
    <logger level="ERROR" message="Logging error: " ++ payload.errorMessage />
  </when>
  <otherwise>
    <logger level="INFO" message="Processing completed successfully" />
  </otherwise>
</choice>
```

★ Logger does not impact the flow. It is a non-blocking component and has no output.

INTERVIEW QUESTIONS

1. What is Logger component in Mule?
2. What are the different log levels available in Mule?
3. How does Logger help in debugging?
4. Can Logger change the message payload?
5. How can you log dynamic content using Logger?
6. How can you control Logger levels?
7. How does Logger help in production environments?

QUICK REFERENCE

Element	Syntax / Example
Logger Component	<pre><logger level="INFO" message="Hello" /></pre>
Log with Variable	<pre><logger level="DEBUG" message="ID: " ++ vars.id /></pre>
Log Error	<pre><logger level="ERROR" message="Error: " ++ vars.error /></pre>
Log with Condition	Use <code><when></code> or <code><otherwise></code> for conditional logging.
Log File Location	<code>\${MULE_HOME}/logs/<app-name>.log</code>

KEY TAKEAWAYS

- ★ Logger component is used to log messages without impacting the flow.
- ★ Choose proper log levels for effective monitoring and debugging.
- ★ Use DataWeave for dynamic and meaningful log messages.
- ★ Avoid logging sensitive data and unnecessary details.
- ★ Logger is essential for building maintainable and production-ready applications.



Use Logger wisely to make your Mule applications easy to **monitor**, **debug** and **support**!



5.6 TRANSFORM MESSAGE

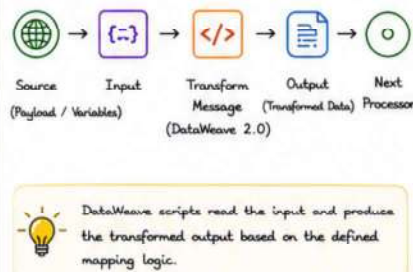


Transform Message component is used to transform the payload, attributes or variables using **DataWeave 2.0**. It is the most powerful way to **shape** and **map** data in Mule.

WHAT IS TRANSFORM MESSAGE?

- ✓ Uses DataWeave 2.0 for data transformation.
- ✓ Transforms payload, attributes or variables.
- ✓ Supports complex mappings, functions, filtering and formatting.
- ✓ Preferred over Set Payload for non-trivial transformations.

HOW IT WORKS



BASIC SYNTAX

```
%dw 2.0
output application/json
---
{
  key: value
  // use input payload
  otherKey: payload.id
}
```

Element	Description
%dw 2.0	Main Transform Message component.
output	Defines the output format.
---	Separates header from body.
payload	Refers to the input payload.

INPUT SOURCES IN DATAWEAVE

Input Object	Description
payload	The current message payload.
attributes	Attributes of the Mule event.
vars	Variables stored in the event.
flowVars	Flow variables.
sessionVars	Session variables (if used).
Mule	Mule functions.
Mule::uuid()	Generates UUID.

TRANSFORM MESSAGE EXAMPLES

- Simple Payload Transformation**

```
%dw 2.0
output application/json
---
{
  id: payload.id,
  name: payload.name,
  amount: payload.price as Number
}
```
- Filtering and Mapping**

```
%dw 2.0
output application/json
---
payload orders map (item) -> {
  orderId: item.id,
  total: item.price * item.qty
}
```
- Using Variables**

```
%dw 2.0
output application/json
---
{
  user: vars.currentUser.id,
  orderCount: sizeOf(payload.orders)
}
```
- Creating Nested Structure**

```
%dw 2.0
output application/json
---
{
  customer: {
    id: payload.id,
    details: payload.details
  }
}
```

SET ATTRIBUTES USING DATAWEAVE

```
<set-attributes value="#[{
  'correlationId': uuid(),
  'timestamp': now()
}]">
doc:name="Set Attributes Example"/>
-----
%dw 2.0
output application/json
---
{
  correlationId: attributes.correlationId,
  timestamp: now()
}
// use in next steps
```

TRANSFORM MESSAGE IN A FLOW



Sample DataWeave Script

```
%dw 2.0
output application/json
---
{
  orderId: payload.id,
  customer: payload.customer.name,
  totalAmount: payload.amount,
  currency: payload.currency default "USD"
}
```

Input Payload

```
{
  "id": "ORD123",
  "customer": { "name": "John Doe" },
  "amount": 250,
  "currency": "USD", "qty": 2
}
```

Output Payload

```
{
  "orderId": "ORD123",
  "customer": "John Doe",
  "totalAmount": 250,
  "currency": "USD"
}
```

Key Points

- ✓ DataWeave script to format output structure.
- ✓ Supports complex mapping, functions and expressions.
- ✓ Keeps flows short and reusable.
- ✓ Powerful with DataWeave 2.0 capabilities.

BEST PRACTICES

- ✓ Use meaningful names for Transform component.
- ✓ Keep DataWeave scripts modular and reusable.
- ✓ Use variables for dynamic values.
- ✓ Handle null and defaults using default operator.
- ✓ Validate input before transforming.
- ✓ Add comments in DataWeave for complex logic.
- ✓ Prefer Transform Message over multiple Set Payloads.

COMMON USE CASES

- ✚ Data format conversion (JSON ↔ XML ↔ CSV)
- ✚ API request/response transformation
- ✚ Data enrichment (add, remove, rename fields)
- ✚ Filtering and aggregation
- ✚ Creating complex structures
- ✚ Mapping between systems

ERROR HANDLING

If DataWeave script has an error, it throws an exception.

Handle it using Error Handler.



INTERVIEW QUESTIONS

1. What is the purpose of Transform Message component?
2. What language is used for transformation?
3. How is Transform Message different from Set Payload?
4. What are input sources in DataWeave?
5. How do you use variables in DataWeave?
6. How do you handle null values in DataWeave?

QUICK REFERENCE

Item	Use
payload	Access message payload
attributes	Access message attributes
vars	Access variables
now()	Current date & time
uuid()	Generate unique ID
sizeOf()	Get size of array/string



Transform Message is the heart of **data shaping** in Mule.
Master **DataWeave**, master integration!



5.7 SET PAYLOAD



Set Payload component is used to **set or replace** the current message **payload** with a static value, **DataWeave expression**, **variable**, or the result of a function.

WHAT IS SET PAYLOAD?

- ✓ Sets or replaces the current message payload.
- ✓ Accepts static values, expressions, variables or functions.
- ✓ Useful for initializing, formatting or preparing data for next steps.
- ✓ Does not change attributes or variables.
- ✓ Important component in any Mule flow.

HOW SET PAYLOAD WORKS



Set Payload takes the provided value or expression and makes it the new payload for the event.

BASIC SYNTAX

```

<set-payload value="{expression}"
  mediaType="application/json" />
or
<set-payload>
  <set-payload value="{expression}" />
</set-payload>
  
```

Attribute	Description
value	The value or DataWeave expression to set as payload.
mediaType (optional)	MIME type of the payload (default: application/json).

TYPES OF PAYLOAD YOU CAN SET

Type	Description
Static Value	"Hello Mule"
DataWeave Expression	<code>#[payload.name]</code>
Variable	<code>#[vars.var1]</code>
Function / Operation	<code>#[upper(payload)]</code>
Object / Array / JSON	<code>#[{ "id": 101, "name": "John" }]</code>

SET PAYLOAD EXAMPLES (DATAWEAVE 2.0)

- Set Static String**
`<set-payload value="Hello Mule" mediaType="text/plain" />`
- Set Number**
`<set-payload value="100" mediaType="application/json" />`
- Set Using DataWeave Expression**
`<set-payload value="#[payload.user.name]" />`
- Set JSON Object**
`<set-payload value="#[%dw 2.0
 output application/json

 {
 status: 'SUCCESS',
 message: 'Payload set successfully'
 }]" />`

SET PAYLOAD IN A FLOW



Example: DataWeave Payload

```

%dw 2.0
output application/json
---
{
  "id": payload.id,
  "name": payload.name,
  "status": "SUCCESS"
}
  
```

COMMON USE CASES

- ✓ Initializing payload with default values.
- ✓ Creating response structures.
- ✓ Preparing payload for next formats.
- ✓ Mapping payload for REST/API calls.
- ✓ Replacing current data with computed results.
- ✓ Returns custom response to client.

SET PAYLOAD WITH DIFFERENT DATA TYPES

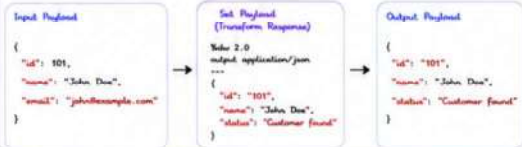
Data Type	Example Expression	Result
String	<code>"MuleSoft Developer"</code>	<code>"MuleSoft Developer"</code>
Number	<code>100</code>	<code>100</code>
Boolean	<code>true</code>	<code>true</code>
Array	<code>#[1,2,3,4]</code>	<code>[1,2,3,4]</code>
Object	<code>#[{name: "John", age: 30}]</code>	<code>{ "name": "John", "age": 30 }</code>
Null	<code>null</code>	<code>null</code>

BEST PRACTICES

- ✓ Use DataWeave expressions for dynamic values.
- ✓ Keep payload small and structured.
- ✓ Use meaningful names for variables.
- ✓ Avoid hardcoding large JSON in flows.
- ✓ Validate payload before sending to external systems.

REAL WORLD EXAMPLE

Scenario: API response formatting



SET PAYLOAD vs TRANSFORM MESSAGE

Aspect	Set Payload	Transform Message
Purpose	Replace payload only	Transform payload, attributes or variables
Attributes Support	No	Yes
Use Case	Simple payload update or initialization	Complex mapping and transformation
Performance	Slightly lighter	Slightly heavier
Best For	Quick payload changes	Complete data transformation

ERROR HANDLING

Set Payload throws an error in case the expression is invalid.

Example Error (Invalid Expression):

```
<set-payload value="#[payload.name.first]" />
```

Error:

EXPRESSION: Cannot resolve property 'name' on null



INTERVIEW QUESTIONS

1. What is the purpose of Set Payload component?
2. Can Set Payload replace the current payload?
3. What data types can be set as payload?
4. How do you set a JSON object as payload?
5. Difference between Set Payload and Transform Message?
6. When would you use Set Payload in a flow?
7. What happens if the expression in Set Payload is invalid?

QUICK REFERENCE

Concept	Syntax / Example
Set static string	<code><set-payload value="Hello" /></code>
Set number	<code><set-payload value="100" /></code>
Set using variable	<code><set-payload value="#[vars.var1]" /></code>
Set JSON object	<code><set-payload value="#[{id: 101, name: 'John'}]" /></code>
Set using DataWeave	<code><set-payload value="#[upper(payload.name)]" /></code>
Set null	<code><set-payload value="null" /></code>



Use Set Payload to **shape**, **initialize** and **control** your **data flow** effortlessly!



5.8 SET VARIABLE



Set Variable component is used to **store data in variables** for later use in the same flow or even across flows. It helps in **maintaining state** and ensuring **performance**.

WHAT IS SET VARIABLE?

- ✓ Creates or updates a variable with specified value.
- ✓ Variables can be used anywhere in the same flow.
- ✓ Helps in storing information temporarily.
- ✓ Does not modify the payload.
- ✓ Supports DataWeave expressions for dynamic values.

HOW SET VARIABLE WORKS



The variable is stored in the event and can be accessed using `vars.variableName`.

BASIC SYNTAX

```

<set-variable variableName="varName"
  value="#(expression)" />
or
<set-variable variableName="varName"
  value="staticValue" />
  
```

Where,

Attribute	Description
variableName (required)	Name of the variable.
value (required)	Value or expression to assign to the variable.

EXAMPLE USAGE

1. Set a static value

```
<set-variable variableName="status" value="SUCCESS" />
```
2. Set a value using DataWeave expression

```
<set-variable variableName="orderCount" value="#{sizeOf(payload.orders)}" />
```
3. Set a variable using function result

```
<set-variable variableName="currentTime" value="#{now()}" />
```

ACCESSING VARIABLES

Variables are accessed using the `vars` object.

- `vars.variableName` // Use in expressions
- `vars.variableName` // Use in DataWeave

Example in Transform Message

```

Idw 2.0
output application/json
---
{
  orderId: vars.orderId,
  status: vars.status,
  time: vars.currentTime
}
  
```

SET VARIABLE IN A FLOW



Example: Store Support Info

```

Idw 2.0
output application/json
---
{
  "id": payload.id,
  "name": payload.name,
  "status": vars.status
}
  
```

TYPES OF VARIABLES

Type	Description
Flow Variable	Lives within the flow.
Session Variable	Lives across multiple flows (if sessionVars used).
Inbound Properties	Read-only properties from the inbound request.

VARIABLE LIFECYCLE

- ✓ Created or updated when Set Variable is executed.
- ✓ Lives in the event until the flow completes.
- ✓ Not visible outside the flow unless shared (e.g., in Object Store or external system).
- ✓ Lost when the flow execution ends.

BEST PRACTICES

- ✓ Use meaningful and consistent variable names.
- ✓ Avoid storing large payloads in variables.
- ✓ Use variables for frequently used data.
- ✓ Clear variables if not needed further.
- ✓ Prefer Object Store for cross-flow persistence.

DATAWEAVE USAGE WITH VARIABLES

1. Using Variable in Expression

```
Idw 2.0
output application/json
---
{
  "userId": vars.userId,
  "orderCount": vars.orderCount
}
  
```
2. Using Variable in Flow

```
Idw 2.0
output application/json
---
{
  "createdAt": vars.currentTime,
  "status": vars.status
}
  
```
3. Using Variable as Condition

```
Idw 2.0
output application/json
---
{
  "isActive": vars.status == "SUCCESS"
}
  
```

COMMON USE CASES

- Store IDs from requests for later use
- Save configuration values
- Share results between steps
- Maintain flags or counters
- Pass calculated values to subsequent steps

REAL WORLD EXAMPLE

Scenario: Get customer details and status flag



SET VARIABLE vs SET PAYLOAD

Aspect	Set Variable	Set Payload
Purpose	Store data for later use	Replace entire payload
Affects Payload	No	Yes
Use Case	Share data inside flow	Transform or change output
Performance	Lightweight	Heavy if large payload
Best For	Temporary data storage	Complete data transformation

ERROR HANDLING

Set Variable does not throw an error. Errors occur in the expression.
Example Error:

```
<set-variable value="#{payload.name.first}" />
```

Error:
 EXPRESSION: Cannot resolve property 'first' on null



INTERVIEW QUESTIONS

1. What is Set Variable component in Mule?
2. How does Set Variable differ from Set Payload?
3. Where are variables stored in Mule?
4. How do you access variables in DataWeave?
5. Can variables persist across flows?

QUICK REFERENCE

Action	Syntax / Example
Set static value	<code><set-variable variableName="status" value="SUCCESS" /></code>
Set using expression	<code><set-variable variableName="count" value="#{sizeOf(payload)}" /></code>
Access variable	<code>vars.variableName</code>
Set using function	<code><set-variable variableName="currentTime" value="#{now()}" /></code>

KEY TAKEAWAYS

- ★ Use Set Variable to store data and reuse it in the flow.
- ★ Variables help maintain state and improve performance.
- ★ Keep variables small and meaningful.
- ★ Use clear names and good practices for maintainable flows.



Use Set Variable to **store**, **reuse** and **control** data in your Mule flows efficiently!



5.9

MULE MESSAGE STRUCTURE

from salesforce

WHAT IS MULE MESSAGE?

- Mule Message is the data container used throughout a Mule Flow.
- It carries all the information required during processing.
- Every Mule event contains a Mule Message.
- Mule Message is immutable (changes create a new message internally).

MULE MESSAGE COMPONENTS

HOW MULE MESSAGE FLOWS

MESSAGE STRUCTURE OVERVIEW

PAYLOAD

Stores the actual business data.

Examples:

- JSON
- XML
- CSV
- Java Objects
- Text Files
- Binary Data

ATTRIBUTES

Stores metadata related to the message.

Examples:

- HTTP Method
- Query Parameters
- URI Parameters
- Headers
- File Information
- Status Codes

VARIABLES

Used to store temporary values.

Examples:

- Customer ID
- Order Number
- Authentication Token
- Processing Flags
- Calculated Values

COMMON OPERATIONS

Component	Purpose
Payload	Stores actual data
Attributes	Stores metadata
Variables	Stores temporary information
Transform Message	Modifies payload
Set Variable	Creates variables
Logger	Logs message details

PAYLOAD EXAMPLES

JSON Payload
 Customer Details

```
{ }
```

XML Payload
 Product Information

```
</>
```

CSV Payload
 Employee Records

```
1 2 3
```

Java Object
 Application Data

Binary Payload
 PDF / Image Files

ATTRIBUTES EXAMPLES

VARIABLES USE CASES

- Store API responses.
- Maintain state across processors.
- Pass values between components.
- Store temporary calculations.
- Improve reusability of data.

REAL WORLD SCENARIOS

API Integrations

Store:

- Customer Information
- Order Details
- Payment Information

Salesforce Integrations

Store:

- Account IDs
- Contact Details
- Access Tokens

Database Operations

Store:

- Query Results
- Generated IDs
- Processing Status

INTERVIEW QUESTIONS

- What is Mule Message?
- What are the components of Mule Message?
- What is the difference between Payload and Attributes?
- What are Variables used for?
- Is Mule Message mutable or immutable?
- How is Mule Message processed inside a Mule Flow?
- Where should temporary values be stored?

QUICK REFERENCE

Component	Usage
Payload	Business Data
Attributes	Metadata
Variables	Temporary Values
Transform Message	Data Transformation
Set Variable	Store Values
Logger	Debugging

KEY TAKEAWAYS

- Mule Message is the heart of every Mule application.
- It consists of Payload, Attributes and Variables.
- Payload stores business data.
- Attributes store metadata.
- Variables store temporary values.
- Understanding Mule Message is essential for MuleSoft interviews and real-world integrations.

"Master Mule Message Structure to build clean, scalable and maintainable MuleSoft integrations."

5.10

CHOICE ROUTER

MuleSoft
from salesforce

Choice Router routes the flow to **one** of the multiple routes based on the **first** condition that evaluates to **true**.
If no condition matches, the **default** route is executed (if defined).

WHAT IS CHOICE ROUTER?

- ★ Evaluates conditions in sequential order.
- ★ Routes to the first matching condition.
- ★ Supports multiple when conditions and one otherwise (default) route.
- ★ Useful for decision making based on message content.
- ★ Only one route is executed per Choice.

HOW CHOICE ROUTER WORKS

Conditions are evaluated in order.
Only the first condition that is **true** is executed.

BASIC SYNTAX

```
<choice>
  <when expression = "#{expression1}">
    <!-- Route for condition 1 -->
  </when>
  <when expression = "#{expression2}">
    <!-- Route for condition 2 -->
  </when>
  ...
  <otherwise>
    <!-- Default route -->
  </otherwise>
</choice>
```

Element	Description
choice	Defines the Choice router.
when	Defines a condition and its route.
expression	DataWeave expression that returns boolean.
otherwise	Default route if no condition matches.

CHOICE ROUTER IN A FLOW

EXAMPLE FLOW SNIPPET

"Flow: order-processing-flow"

- When payload.type = "EXPRESS"
→ Route to "Processing Express Order"
- When payload.type = "STANDARD"
→ Route to "Processing Standard Order"
- When payload.priority = "HIGH"
→ Route to "Processing High Priority Order"
- Otherwise
→ Route to "Processing Default Order"

EXAMPLE PAYLOADS & RESULTS

Payload	Evaluation	Route Executed
{ "type": "EXPRESS" }	payload.type == "EXPRESS" = true	Route 1
{ "type": "STANDARD" }	First = false Second = true	Route 2
{ "priority": "HIGH" }	First = false Second = false Third = true	Route 3
{ "type": "BULK" }	All conditions false	Default Route

DATAWEAVE EXPRESSION EXAMPLES

- #{payload.type == "EXPRESS"}
- #{payload.amount > 1000}
- #{attributes.queryParams.action == "create"}
- #{vars.isValid == true}
- #{sizeOf(payload.items) > 0}

CONFIGURATION OPTIONS

Option	Description
Order of when conditions	Conditions are evaluated top to bottom.
Otherwise (default)	Executed only if none of the when conditions are true.
Expression Type	Must return boolean (true/false).
Route Execution	Only one route is executed per Choice.

BEST PRACTICES

- ✓ Order conditions from most specific to least specific.
- ✓ Keep expressions simple and efficient.
- ✓ Always define an otherwise route for unexpected cases.
- ✓ Use meaningful logging in each route.
- ✓ Avoid overlapping conditions.

COMMON USE CASES

- Routing based on message type.
- Handling different customer tiers.
- Processing based on priority or status.
- Conditional flow for system calls.
- Data validation and branching logic.

MONITORING & LOGGING

- ♥ Log which route is executed.
- ♥ Monitor condition hits.
- ♥ Use custom metrics for analysis.
- ♥ Alert on unexpected default route executions.

ERROR HANDLING

Handle errors within each route using error-handler.

- Use On Error Continue / Propagate as required.

INTERVIEW QUESTIONS

1. What is Choice Router in MuleSoft?
2. How does Choice Router evaluate conditions?
3. What happens if multiple conditions are true?
4. What is the purpose of otherwise route?
5. How do you debug Choice Router logic?

QUICK REFERENCE

Task	Syntax / Example
Choice start	<choice>
When condition	<when expression="true">
Otherwise (default)	<otherwise>
End choice	</choice>

KEY TAKEAWAYS

- ★ Choice Router makes smart routing decisions.
- ★ First matching condition wins.
- ★ Always include a default route.
- ★ Helps build flexible and efficient Mule flows.

Use Choice Router to make the right decisions and build intelligent, efficient, and scalable Mule applications!

5.11

SCATTER GATHER


MuleSoft
from salesforce



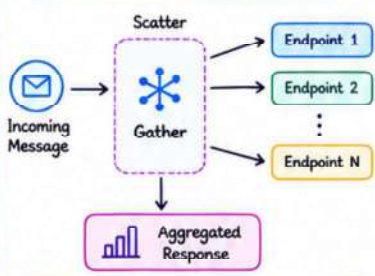
Scatter Gather allows you to send a message to multiple endpoints in parallel (**scatter**) and aggregate their responses (**gather**) into a single result.

It improves performance by **parallel processing** and combines results efficiently.

WHAT IS SCATTER GATHER?

- ✔ Sends a request to multiple endpoints concurrently.
- ✔ Waits for all responses and aggregates them.
- ✔ Improves performance by parallel execution.
- ✔ Supports configurable timeout and fail strategy.
- ✔ Useful for calling multiple services and combining results.

HOW SCATTER GATHER WORKS



★ Requests are sent in parallel and responses are collected and merged.

BASIC SYNTAX

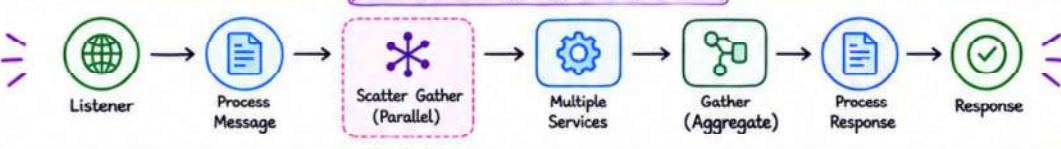
```

<scatter-gather>
<route> <processor-chain/> </route>
<route> <processor-chain/> </route>
...
<aggregator>
  <collection-size>#[vars.size]</collection-size>
  <timeout>5000</timeout>
  <fail-strategy>FAIL</fail-strategy>
  <expression>#[payload]</expression>
</aggregator>
</scatter-gather>

```

Element	Description
route	Defines a route to execute in parallel.
aggregator	Aggregates all route responses.
collection-size	Number of expected responses.
timeout	Max time to wait for responses.
fail-strategy	Action if any route fails.

SCATTER GATHER IN A FLOW



EXAMPLE FLOW SNIPPET

Routes execute in parallel:

1. Call Customer Service
2. Call Order Service
3. Call Inventory Service

↳ Responses are combined into a single payload.

AGGREGATOR EXPRESSION EXAMPLES

Expression	Result
#[payload]	List of all payloads
#[payload joinBy " , "]	Comma separated string
#[payload map (item) -> item.id]	List of IDs
#[payload reduce (acc, item) -> acc + item.total]	Sum of all totals

FAIL STRATEGY OPTIONS

Strategy	Description
FAIL	Fail if any route fails.
IGNORE	Ignore failed routes.
RETURN_PARTIAL	Return successful responses only.

CONFIGURATION OPTIONS

Option	Description
maxConcurrency	Max parallel calls.
timeout	Max wait time for responses.
collectionSize	Expected number of responses.
routingStrategy	ROUND_ROBIN or FIRST_COMPLETED.

BEST PRACTICES

- ✔ Use for independent service calls.
- ✔ Set appropriate timeouts.
- ✔ Choose the right fail strategy.
- ✔ Limit max concurrency.
- ✔ Keep aggregation logic simple.

COMMON USE CASES

- ★ Calling multiple services in parallel
- ★ Aggregating data from multiple sources
- ★ Improving response time of composite APIs
- ★ Dashboard or report data aggregation
- ★ Collecting results from external systems

INTERVIEW QUESTIONS

1. What is Scatter Gather in MuleSoft?
2. How does it improve performance?
3. What are the fail strategy options?
4. What is the difference between ROUND_ROBIN and FIRST_COMPLETED?
5. How do you aggregate responses?

QUICK REFERENCE

Task	Syntax / Example
Scatter start	<scatter-gather>
Add a route	<route>...</route>
Set timeout	<timeout>5000</timeout>
Set fail strategy	<fail-strategy>FAIL</fail-strategy>
End scatter	</scatter-gather>

KEY TAKEAWAYS

- ✔ Enables parallel execution of routes.
- ✔ Combines multiple responses efficiently.
- ✔ Provides better performance & scalability.
- ✔ Use with proper timeouts and fail strategy.
- ✔ Ideal for composite and integration scenarios.



Use Scatter Gather to make the **right decisions** and build **efficient, scalable** and **high performance** Mule applications!



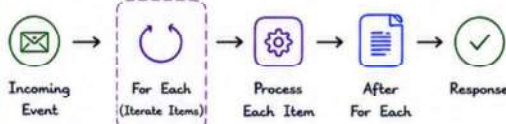
5.12.1 FOR EACH



For Each scope is used to **iterate** over a **collection** (Array, List, Number, Cursor, etc.) and process each item **sequentially**.

It improves **reusability** and keeps flows simple and easy to maintain.

HOW FOR EACH WORKS



The For Each scope iterates through each item in the collection, executes the enclosed processors, and continues until all items have been processed.

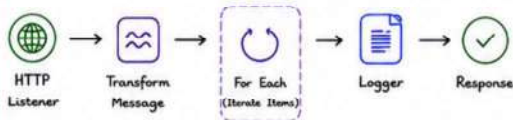
BASIC SYNTAX

```

<for-each collection="#[payload]" item="item" index="index">
  <!-- Process Each Item -->
  <logger level="INFO" message="Processing #[item]
    at index #[index]"/>
  <!-- More Processors -->
</for-each>
  
```

Attribute	Description
collection (required)	The collection to iterate over.
item (optional)	Name of the variable that holds the current item. Default: "item"
index (optional)	Name of the variable that holds the current index. Default: "index"
varName (optional)	Name of the For Each scope.

FOR EACH IN A FLOW



Example Flow Snippet

```

<for-each collection="#[payload]" item="item" index="index">
  <logger level="INFO" message="Item #[item] at index #[index]"/>
  <!-- Process each item -->
</for-each>
  
```

AVAILABLE VARIABLES INSIDE FOR EACH

Variable	Description
item (default)	The current item in the collection.
index (default)	The current index (starting from 0).
vars	All user-defined variables.
vars.size	Total number of items in the collection.
vars.current	Reference flag for For Each status.

COLLECTION TYPES SUPPORTED

- ✓ Array
- ✓ List
- ✓ Set
- ✓ Map (iterator over values)
- ✓ Stream
- ✓ Cursor (with streaming)

Note: If the payload is not a collection, MuleSoft treats it as a single item collection and executes the scope once.

OUTPUT BEHAVIOR

By default, For Each returns a collection of results produced by each iteration.

Scenario	Result
Inner processor returns a value	List of all returned values.
Inner processor does not return payload	Original payload is returned.
Inner processor returns null	List of nulls is returned.
No. of input payloads match for Each	Overrides item count.

EXAMPLE

Input:
["A", "B", "C"]
Each item processed
↓
Output:
["Result A", "Result B", "Result C"]

BEST PRACTICES

- ✓ Use meaningful variable names (item, index).
- ✓ Prefer streaming mode for large collections.
- ✓ Avoid heavy operations inside the loop.
- ✓ Handle error scenarios inside the loop.
- ✓ Keep inner flow lightweight.

COMMON USE CASES

- Processing items from arrays or lists.
- Calling APIs for each item.
- Sending emails or validating each record.
- Inserting or updating multiple records.

INTERVIEW QUESTIONS

1. What is the purpose of For Each in MuleSoft?
2. How does For Each handle incoming payload?
3. What variables are available inside For Each?
4. What is the difference between For Each and Map?
5. What are the collection types supported?

KEY TAKEAWAY

★ For Each helps you iterate over collections sequentially, process each item, and optionally collect results.

Use it to build simple, reusable and efficient Mule flows!



Use For Each to **iterate**, **process**, and **achieve more** – one item at a time!



5.13 PARALLEL FOR EACH



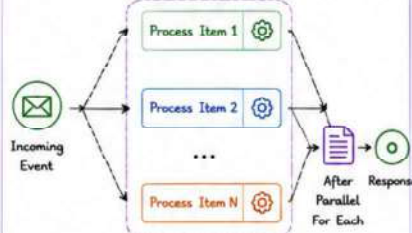
Parallel For Each scope processes items in a collection **concurrently**.

It improves **performance** by leveraging **multiple threads**, while keeping the flow logic **simple and reusable**.

WHAT IS PARALLEL FOR EACH?

- ✔ Iterates over each item in a collection.
- ✔ All items are processed concurrently.
- ✔ Each item is handled in a separate thread.
- ✔ Improves throughput and reduces overall time.
- ✔ Maintains variables like index and size.
- ✔ Returns a collection of results once all threads complete.
- ✔ Best for independent, non-blocking operations.

HOW PARALLEL FOR EACH WORKS



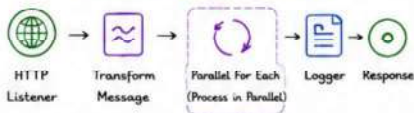
All items are processed concurrently. Once all threads complete, results are aggregated and the flow continues.

BASIC SYNTAX

```
<parallel-for-each collection="#[payload]"
  maxConcurrency="value"
  waitForAll="true">
  <!-- Process / Components -->
</parallel-for-each>
```

Attribute	Description
collection	The collection to iterate over.
maxConcurrency	Max number of threads to run in parallel.
waitForAll	Wait until all threads complete.
maxConcurrency (optional)	Default number of threads = 2

PARALLEL FOR EACH IN A FLOW



Sample Flow Snippet

```
<parallel-for-each collection="#[payload.items]"
  maxConcurrency="5" waitForAll="true">
  <!-- Process each item in parallel -->
</parallel-for-each>
```

AVAILABLE VARIABLES INSIDE PARALLEL FOR EACH

Variable	Description
item (default)	The current item in the collection.
index (default)	The current index (starting from 0).
vars	All user-defined variables.
vars.size	Total number of items in collection.
vars.current	Reference flag for Parallel For Each.

Example Usage

- item.id → Current item
- index → Current index
- vars.size → Total items

COLLECTION TYPES SUPPORTED

Parallel For Each works with:

- ✔ Array
- ✔ List
- ✔ Set
- ✔ Map (Iterator over values)
- ✔ Iterator
- ✔ Cursor (streaming supported)

Note:

If payload is not a collection, MuleSoft treats it as a single item collection and executes the scope once.

CONFIGURATION OPTIONS

Attribute	Description
maxConcurrency	Limits the number of threads in parallel.
timeout	Max time to wait for all threads.
failStrategy	Action when a thread fails.
maxConcurrency (default)	Default threads = 2

RESULTS AGGREGATION

By default, results from each iteration are aggregated into a List.

Example Output (Aggregated Result):

```
[ "Item 1", "Item 2", "Item 3",
  "Item 4", "Item 5" ]
```

If any thread fails, overall result depends on the configured failStrategy.

STREAMING SUPPORT

For large collections, enable streaming to avoid loading all items in memory.

```
<parallel-for-each collection="#[payload]"
  streaming="true" maxConcurrency="5">
  <!-- Process items in streaming mode -->
</parallel-for-each>
```

Streaming is recommended for large datasets or low memory scenarios.

EXAMPLES

1. Iterate Over Array

Input: ["A", "B", "C"]
Output: List of results
(Each item processed in parallel)



2. Iterate With Index

Input: [10, 20, 30]
Output: index → 0,1,2
item → 10,20,30



3. Iterate Over Map

Input: {"a":1, "b":2}
Output: Processes values
(1, 2) in parallel



4. Streaming Large Data

Input: Large Collection
Output: Memory efficient
streaming



5. Iterate Over Cursor

Input: Database Cursor
Output: Records processed
concurrently



COMMON USE CASES

- ✔ Processing large datasets.
- ✔ Calling multiple APIs in parallel.
- ✔ Data aggregation.
- ✔ Batch processing.

BEST PRACTICES

- ✔ Use meaningful variable names.
- ✔ Keep operations independent.
- ✔ Prefer streaming for large data.
- ✔ Handle errors in each thread.

ERROR HANDLING

Handle errors inside the Parallel For Each scope to avoid stopping the entire flow.



PARALLEL vs FOR EACH

Aspect	For Each	Parallel For Each
Execution	Sequential	Concurrent
Performance	Slower	Faster
Use Case	Small Data	Large Data
Threads	Single Thread	Multiple Threads

KEY TAKEAWAY

Parallel For Each helps you process collections faster by using multiple threads. Use it for better performance and scalability!



Use Parallel For Each to process **more**, in **less time** - safely and efficiently!



5.14

UNTIL SUCCESSFUL

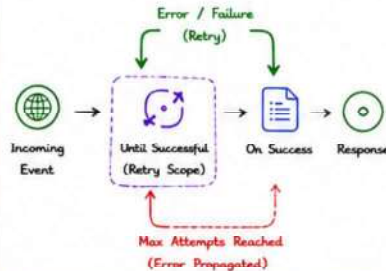


Until Successful scope **repeatedly** executes the enclosed processor until it completes successfully, fails max retries or until the maximum number of attempts is reached. It is ideal for handling **transient errors**.

WHAT IS UNTIL SUCCESSFUL?

- ✓ Retries the enclosed processor until it succeeds.
- ✓ Handles transient errors automatically.
- ✓ Configurable max attempts and retry interval between tries.
- ✓ Optional exponential backoff for increasing delays.
- ✓ If all attempts fail, the error is propagated.

HOW UNTIL SUCCESSFUL WORKS



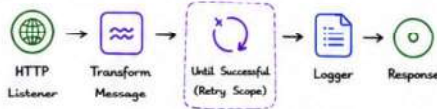
The enclosed processor is executed until it succeeds or the maximum number of attempts is reached.

BASIC SYNTAX

```
<until-successful maxRetries="3"
  millisBetweenRetries="2000"
  exponentialBackoff="true">
  <!-- Flow / Components -->
  <processor .... />
  <error-handler ref="POST_path" />
</until-successful>
```

Attribute	Description
maxRetries (required)	Maximum number of retry attempts.
millisBetweenRetries (required)	Wait time between retries in milliseconds.
exponentialBackoff (optional)	Enable exponential backoff (true/false).
backoffFactor (optional)	Multiplier for wait time when backoff is enabled.

UNTIL SUCCESSFUL IN A FLOW



Sample Flow Snippet

```
<until-successful maxRetries="3" millisBetweenRetries="2000"
  exponentialBackoff="true" backoffFactor="2.0">
  <!-- Your processors here -->
  <error-handler ref="POST_path" />
</until-successful>
```

CONFIGURATION OPTIONS

Option	Description
maxRetries	Number of retry attempts after failure.
millisBetweenRetries	Initial wait time between retries (in ms).
exponentialBackoff	Enables increasing wait time after each retry.
backoffFactor	Multiplier for wait time when backoff is enabled.

Example (BackoffFactor = 2.0)

- Attempt 1 → Wait 2000 ms
- Attempt 2 → Wait 4000 ms
- Attempt 3 → Wait 8000 ms
- Attempt 4 → Wait 16000 ms
- ...

HOW RETRIES WORK

- 1 Enclosed processor is executed.
- 2 If an error occurs, retrying starts after wait time.
- 3 If it succeeds, Mule exits the scope.
- 4 If retries exceed maxRetries, error is thrown.
- 5 Response is returned.

ERROR HANDLING

If all retry attempts fail, the original error is raised and can be handled by **error-handler**. Use **<error-handler>** inside the scope.

COLLECTION TYPES SUPPORTED

Until Successful can be used with:

- ✓ Array (fixed size)
- ✓ List
- ✓ Set
- ✓ Map (fixed size)
- ✓ Object Store
- ✓ Custom business logic

COMMON USE CASES

- 1 Retrying external API calls.
- 2 Handling temporary database connectivity issues.
- 3 Dealing with network timeouts.
- 4 Processing messages with transient failures.
- 5 Importing/validating external data.

BEST PRACTICES

- ✓ Use for transient errors only.
- ✓ Set appropriate maxRetries and wait times.
- ✓ Use exponential backoff to reduce load.
- ✓ Log each attempt for better observability.

OUTPUT BEHAVIOR

On Success

The result of the successful execution is returned.

On Failure

The error from the last attempt is propagated.

EXAMPLES

1. Retrying API Call

Retry API call until success or max retries reached.

2. Database Operation

Retry DB operation on deadlock or connection issues.

3. File Processing

Retry file read/write if file is temporarily locked.

4. Message Processing

Retry message processing on transient failures from external systems.

INTERVIEW QUESTIONS

- 1 What is Until Successful in MuleSoft?
- 2 How does Until Successful work?
- 3 What is maxRetries used for?
- 4 What is the use of exponential backoff?
- 5 How is error handled in Until Successful?

QUICK REFERENCE

Item	Purpose
maxRetries	Max retry attempts after failure
millisBetweenRetries	Wait time between retries
exponentialBackoff	Increase wait time after each retry
backoffFactor	Multiplier for backoff wait time

KEY TAKEAWAY

Until Successful helps you handle temporary failures gracefully by retrying automatically until success or until the configured limit. Use it wisely to build reliable and resilient Mule flows!



Use Until Successful to build **resilient** and **reliable** Mule flows!



5.15

ASYNC SCOPE



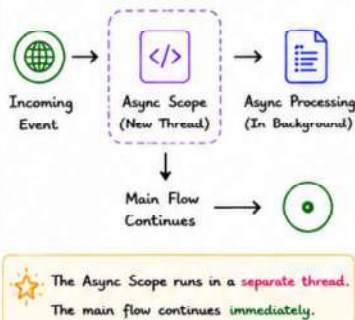
Async Scope processes events **asynchronously** in a separate thread, without blocking the main flow.

It improves **responsiveness** and enables **parallel** execution of independent tasks.

WHAT IS ASYNC SCOPE?

- ✓ Executes a set of processors asynchronously.
- ✓ Does not block the main flow.
- ✓ Improves system responsiveness.
- ✓ Useful for fire-and-forget tasks (logging, notifications, audit trails).
- ✓ Errors can be handled within the scope.

HOW ASYNC SCOPE WORKS



BASIC SYNTAX

```
<async>
  <async-executor name="myExecutor"
    maxThreads="10" />
  <!-- Flow / Components -->
  <logger level="INFO"
    message="Processing
    in async scope" />
</async>
```

Attribute	Description
async-executor	Reference to the async executor configuration.
maxThreads	Maximum number of threads in the pool.

ASYNC SCOPE IN A FLOW



ERROR HANDLING

Handle errors inside Async Scope using on-error-continue to prevent them from affecting the main flow.

```
<async>
  <error-handler>
    <on-error-continue type="ANY">
      <logger level="ERROR"
        message="Async error: #[error.description]" />
    </on-error-continue>
  </error-handler>
  <!-- Processors -->
</async>
```

COMMON USE CASES

- ✓ Sending emails or notifications
- ✓ Writing logs or audit records
- ✓ Updating external systems
- ✓ Triggering reporting or analytics
- ✓ Fire-and-forget operations

EXAMPLE FLOW

```
<flow name="order-processing-flow">
  <http:listener config-ref="HTTP-Listener-config"
    path="/order" />
  <transform message="#[payload]" />
  <async>
    <async-executor name="asyncExecutor"
      maxThreads="10" />
    <logger level="INFO" message="Processing order
      async: #[payload.orderId]" />
    <db:insert config-ref="DB-Config" />
    <email:send to="user@example.com" />
  </async>
  <logger level="INFO" message="Order request accepted" />
</flow>
```

ASYNC EXECUTOR CONFIG EXAMPLE

```
<async:config name="Async-Config" doc:name="Async Config">
  <async:async-executor name="asyncExecutor"
    maxThreads="10"
    keepAliveTime="60"
    queueCapacity="100" />
</async:config>
```

BEST PRACTICES

- ★ Use Async Scope for non-blocking, independent tasks.
- ★ Handle errors within the Async Scope.
- ★ Configure thread pool (maxThreads) based on load.
- ★ Avoid long-running or blocking operations.
- ★ Ensure idempotent operations when needed.

OUTPUT BEHAVIOR

On Success:
Main flow continues and response is returned.

On Failure:
Error is handled inside the Async Scope and does not impact the main flow.

INTERVIEW QUESTIONS

1. What is Async Scope in MuleSoft?
2. How does Async Scope differ from a normal flow?
3. When should you use Async Scope?
4. How are errors handled in Async Scope?
5. How do you configure Async Executor?

QUICK REFERENCE

Component	Purpose
Async Scope	Executes tasks asynchronously (non-blocking)
Async Executor	Manages thread pool for Async Scope
maxThreads	Maximum number of threads in the pool
on-error-continue	Handles errors within Async Scope

Async Scope helps you build responsive, scalable and high-performing Mule applications!



Use Async Scope to run tasks in the **background** and keep your flows **responsive**!



5.16

BATCH PROCESSING



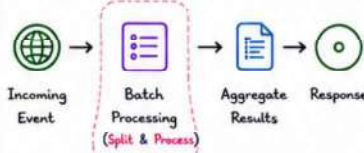
Batch Processing scope divides large payloads into smaller **batches** and processes them in **chunks**.

It improves **performance**, **memory** usage, and **reliability** for bulk data operations.

WHAT IS BATCH PROCESSING?

- ✔ Splits large payloads into smaller batches.
- ✔ Processes each batch sequentially.
- ✔ Reduces memory consumption.
- ✔ Handles bulk data efficiently.
- ✔ Supports error handling, aggregation, and custom batch sizes.

HOW BATCH PROCESSING WORKS



Data is split into batches → Each batch is processed → Results are aggregated → Final response is returned.

BASIC SYNTAX

```

<batch:job jobName="batchJob"
  maxFailedRecords="1">
  <batch:process-records>
    <!-- Process each record -->
  </batch:process-records>
  <batch:aggregator>
    <!-- Aggregate results -->
  </batch:aggregator>
</batch:job>
  
```

Attribute	Description
jobName	Name of the batch job.
maxFailedRecords	Max number of records that can fail. (-1 = no limit)
batchSize	Number of records per batch.

BATCH PROCESSING IN A FLOW



SAMPLE FLOW SNIPPET

```

<flow name="bulk-processing-flow">
  <http-listener config-ref="HTTP_listener_config" path="/processBulk"/>
  <set-payload value="#[payload]" />
  <batch:job jobName="batchJob" maxFailedRecords="1"
    batchSize="100">
    <batch:process-records>
      <logger level="INFO" message="Processing item: #[payload.id]" />
      <!-- Business logic or DB operations -->
    </batch:process-records>
    <batch:aggregator>
      <logger level="INFO" message="Batch processing completed" />
    </batch:aggregator>
  </batch:job>
  <logger level="INFO" message="Bulk processing completed" />
</flow>
  
```

CONFIGURATION OPTIONS

Option	Description
batchSize	Number of records in each batch.
maxFailedRecords	Maximum number of failed records allowed. (-1 = no limit)
continueOnError	Whether to continue processing after an error. (true/false)
target	Target component for batch aggregation.

COMMON USE CASES

- ✔ Processing large CSV or JSON files
- ✔ Bulk database inserts/updates
- ✔ Data migration and ETL jobs
- ✔ Bulk API integrations
- ✔ Mass email or notification processing

ERROR HANDLING

Handle errors using `maxFailedRecords` and `on-error-continue`.

```

<batch:job jobName="batchJob" maxFailedRecords="1">
  <batch:process-records>
    <logger level="ERROR"
      message="Error processing item: #[error.description]" />
  </batch:process-records>
</batch:job>
  
```

BEST PRACTICES

- ☆ Choose an optimal batch size.
- ☆ Use `continueOnError` wisely.
- ☆ Monitor memory and performance.
- ☆ Aggregate results efficiently.
- ☆ Log failed records for troubleshooting.

INTERVIEW QUESTIONS

- 1 What is Batch Processing in MuleSoft?
- 2 How does Batch Processing work?
- 3 What is the purpose of `batchSize`?
- 4 How do you handle errors in Batch Processing?
- 5 How do you aggregate batch results?



KEY TAKEAWAY

Batch Processing helps you handle large volumes of data efficiently by processing them in smaller **chunks**, reducing memory usage and improving **performance**.



Use Batch Processing to handle large data sets **efficiently** and **reliably**!

5.17

PARALLEL FOR EACH

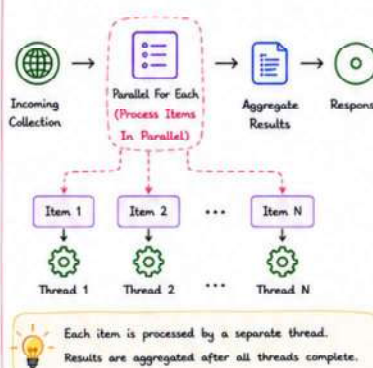


Parallel For Each scope processes each item in a collection **concurrently** using multiple threads. It improves **performance** and reduces overall processing time for **large datasets**.

WHAT IS PARALLEL FOR EACH?

- Processes items in a collection in parallel using multiple threads.
- Reduces total processing time significantly.
- Improves system throughput and responsiveness.
- Ideal for independent item processing.
- Supports error handling and aggregation.
- Configurable concurrency (`maxConcurrency`).

HOW PARALLEL FOR EACH WORKS



BASIC SYNTAX

```
<foreach collection="#[payload.items]">
  <parallel-foreach
    maxConcurrency="5">
    <!-- Process each item in parallel -->
    <logger level="INFO"
      message="Processing item #[item.id]" />
    </parallel-foreach>
    <!-- Aggregate results -->
  </foreach>
```

Attribute	Description
collection	Collection of items to iterate.
maxConcurrency	Maximum number of concurrent threads.
target	Optional target to store aggregated results.
continueOnError	Whether to continue processing remaining items on error.

PARALLEL FOR EACH IN A FLOW



EXAMPLE FLOW SNIPPET

```
<flow name="parallel-foreach-flow">
  <http-listener config-ref="HTTP-Listener-config" path="/processItems" />
  <set-payload value="#[payload.items]" />
  <foreach collection="#[payload.items]">
    <parallel-foreach maxConcurrency="5"
      target="processedItems" continueOnError="true">
      <logger level="INFO" message="Processing item #[item.id]" />
      <!-- Business logic for each item -->
      <set-variable variableName="result" value="#[processItem(item)]" />
    </parallel-foreach>
    <logger level="INFO"
      message="All items processed. Count: #[sizeOf(vars.processedItems)]" />
  </foreach>
</flow>
```

CONFIGURATION OPTIONS

Option	Description
maxConcurrency	Maximum number of threads to process items in parallel.
target	Variable to store the list of results.
continueOnError	true: Continue processing other items on error. false: Stop processing on first error.
timeout	Maximum time to wait for all threads to complete.
waitForCompletion	true: Wait for all threads to finish (default). false: Continue without waiting.

ERROR HANDLING

Use `on-error-continue` or `on-error-propagate` inside the scope.

```
<parallel-foreach maxConcurrency="5">
  <!-- Process item -->
  <try>
    <error-handler>
      <on-error-continue type="ANY">
        <logger level="ERROR"
          message="Error processing item #[item.id]:
            #[error-description]" />
      </on-error-continue>
    </error-handler>
  </try>
</parallel-foreach>
```

COMMON USE CASES

- Processing large lists of records from database or APIs.
- Bulk data transformations or validations.
- Parallel calls to external systems or microservices.
- Generating reports or documents for many items.
- Real-time data enrichment at scale.

BEST PRACTICES

- Choose an optimal `maxConcurrency` based on system capacity.
- Ensure items are independent to avoid data conflicts.
- Handle errors per item to avoid failing the entire batch.
- Avoid heavy operations inside threads; keep them efficient.
- Monitor thread usage and system resources.

INTERVIEW QUESTIONS

- What is Parallel For Each in MuleSoft?
- How does Parallel For Each differ from For Each?
- How do you control the number of threads?
- How are results aggregated in Parallel For Each?
- How do you handle errors in Parallel For Each?

KEY TAKEAWAY

Parallel For Each helps you process items **concurrently**, reducing overall processing time and improving **performance** for large datasets when items are independent.



Use Parallel For Each to process **more** items in less time and build **high-performance** Mule applications!



5.18

VALIDATION MODULE

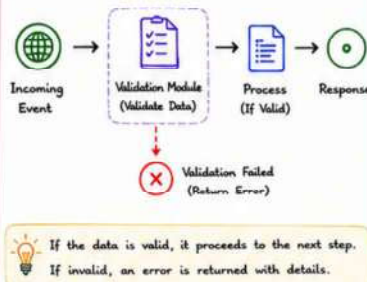


Validation Module scope **validates data** against defined rules or schemas. It ensures **data quality**, prevents invalid data from processing further, and provides clear **error messages**.

WHAT IS VALIDATION MODULE?

- ✓ Validates incoming data against rules or schemas.
- ✓ Supports built-in and custom validation rules.
- ✓ Prevents invalid data from moving forward.
- ✓ Provides meaningful error messages.
- ✓ Easily reusable and configurable.

HOW VALIDATION MODULE WORKS



BASIC SYNTAX

```
<validation:validate
  doc:name="Validate Data"
  config-ref="Validation_Config" />
```

Attribute	Description
doc:name	Name of the validation module.
config-ref	Reference to the validation configuration.

VALIDATION MODULE IN A FLOW



EXAMPLE FLOW SNIPPET

```
<flow name="user-validation-flow">
  <http:listener config-ref="HTTP_Listener_config" path="/user" />
  <set-payload value="#[payload]" />
  <validation:validate doc:name="Validate User"
    config-ref="User_Validation_Config" />
  <logger level="INFO" message="User data is valid." />
  <flow-ref name="process-user-flow" />
  <error-handler>
    <on-error-continue type="VALIDATION:INVALID_INPUT_DATA">
      <logger level="ERROR"
        message="Validation failed: #[error.description]" />
      <set-payload value="{('status': 'error',
        'message': '#[error.description]')}" />
    </on-error-continue>
  </error-handler>
</flow>
```

VALIDATION CONFIGURATION EXAMPLE

```
<validation:config name="User_Validation_Config" doc:name="User Validation Config">
  <validation:rule name="Validate User Payload">
    <validation:all>
      <validation:rule doc:name="Name is Required">
        <validation:not-null value="#[payload.name]" />
      </validation:rule>
      <validation:rule doc:name="Email Format is Valid">
        <validation:expression
          expression="#[payload.email matches '[^@]+@[^.]+\.[^@]+$']" />
      </validation:rule>
      <validation:rule doc:name="Age is Greater than 18">
        <validation:expression expression="#[payload.age > 18]" />
      </validation:rule>
    </validation:all>
  </validation:rule>
</validation:config>
```

ERROR HANDLING

Validation errors are handled using `on-error-continue` or `on-error-propagate`.

```
<error-handler>
  <on-error-continue type="VALIDATION:INVALID_INPUT_DATA">
    <logger level="ERROR"
      message="Validation failed: #[error.description]" />
    <set-payload value="{('status': 'error',
      'message': '#[error.description]')}" />
  </on-error-continue>
</error-handler>
```

COMMON USE CASES

- ✓ Validating API request payloads.
- ✓ Ensuring required fields are present.
- ✓ Checking data formats (email, phone, date, etc.).
- ✓ Verifying business rules (age > 18, amount > 0, etc.).
- ✓ Preventing bad data from entering systems.

BEST PRACTICES

- ★ Keep validation rules simple and focused.
- ★ Use descriptive error messages.
- ★ Reuse validation configurations across flows.
- ★ Fail fast to avoid unnecessary processing.
- ★ Version validation configs for maintainability.

INTERVIEW QUESTIONS

1. What is Validation Module scope in MuleSoft?
2. How does Validation Module work?
3. What are the different validation rules supported?
4. How do you handle validation errors?
5. Can you reuse validation configurations?



KEY TAKEAWAY

Validation Module ensures only valid and **quality data** moves forward in your integration flows, improving **reliability** and **data integrity**.



Use Validation Module to ensure **data quality**, prevent **errors**, and build **reliable integrations**!



5.18

ERROR HANDLING ✨

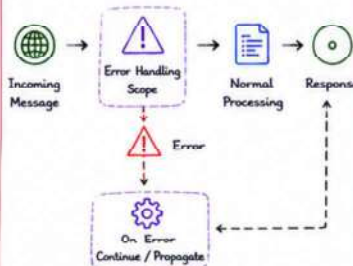


Error Handling scope manages errors that occur during message processing. It allows you to **catch**, **handle**, and **respond** to errors gracefully without disrupting the flow, improving **reliability** and **user experience**.

WHAT IS ERROR HANDLING?

- ✓ Catches errors that occur in a flow.
- ✓ Prevents application crashes and ensures graceful degradation.
- ✓ Returns meaningful error responses to clients.
- ✓ Supports different error types and strategies.
- ✓ Centralizes error handling logic.

HOW ERROR HANDLING WORKS



If an error occurs → it is caught by the Error Handling scope → handled accordingly → flow continues or returns an error response.

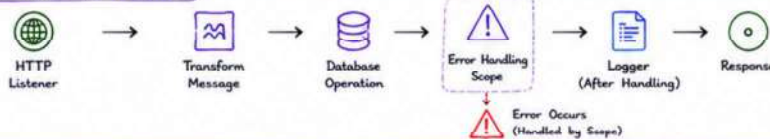
BASIC SYNTAX

```

<error-handler>
  <on-error-continue type="ANY">
    <!-- Handle the error -->
    <logger level="ERROR"
      message="An error occurred: #[error.description]"/>
    <!-- Build custom error response -->
  </on-error-continue>
  <on-error-propagate type="HTTP:NOT_FOUND">
    <!-- Propagate specific errors -->
  </on-error-propagate>
</error-handler>
  
```

Element	Description
on-error-continue	Handles the error and continues processing the flow.
on-error-propagate	Propagates the error to the caller.
type	Error type to catch (e.g., ANY, HTTP:NOT_FOUND).

ERROR HANDLING IN A FLOW



EXAMPLE FLOW SNIPPET

```

<flow name="create-order-flow">
  <http-listener config-ref="HTTP_Listener_config" path="/orders"/>
  <set-payload value="#[payload]"/>
  <error-handler>
    <on-error-continue type="ANY">
      <logger level="ERROR"
        message="Error occurred: #[error.description]"/>
      <set-variable variableName="errorMessage"
        value="#[error.description]"/>
      <set-payload value="#{'status':'error',
        'message': vars.errorMessage}"/>
      <set-property propertyName="httpStatus" value="500"/>
    </on-error-continue>
  </error-handler>
  <db:insert config-ref="DB_Config"/>
  <set-payload value="#{'status':'success'}" mediaType="application/json"/>
</flow>
  
```

ERROR TYPES EXAMPLES

Error Type	Description
ANY	Catches all types of errors.
DEF:VALIDATION	DataWeave validation errors.
HTTP:NOT_FOUND	HTTP 404 errors.
HTTP:BAD_REQUEST	HTTP 400 errors.
DB-CONNECTIVITY	Database connectivity errors.
CUSTOM	Custom application errors.

ERROR HANDLING STRATEGIES

- ✓ Use on-error-continue for recoverable errors.
- ✓ Use on-error-propagate for critical errors.
- ✓ Return meaningful error messages to clients.
- ✓ Log errors for monitoring and troubleshooting.
- ✓ Set appropriate HTTP status codes.

ERROR HANDLING SNIPPET (ON-ERROR-PROPAGATE)

```

<error-handler>
  <on-error-propagate type="HTTP:NOT_FOUND"/>
</error-handler>
  
```

ERROR RESPONSE STRUCTURE (EXAMPLE)

```

{
  "status": "error",
  "message": "Resource not found",
  "errorCode": "404",
  "timestamp": "2024-05-24T10:30:00Z"
}
  
```

COMMON USE CASES

- ✓ Handling system or integration errors.
- ✓ Returning user-friendly error responses.
- ✓ Managing validation and business rule errors.
- ✓ Handling timeouts and connectivity issues.
- ✓ Ensuring graceful degradation of services.

BEST PRACTICES

- ★ Handle specific errors before using ANY.
- ★ Provide clear and consistent error messages.
- ★ Avoid exposing internal error details to clients.
- ★ Log errors with sufficient context.
- ★ Monitor error rates and trends.

INTERVIEW QUESTIONS

- 1 What is Error Handling scope in MuleSoft?
- 2 How does Error Handling scope work?
- 3 What is the difference between on-error-continue and on-error-propagate?
- 4 How do you handle different types of errors?
- 5 How do you return custom error responses in MuleSoft?



KEY TAKEAWAY

Error Handling scope helps you catch, manage, and respond to errors gracefully, ensuring application **reliability** and a better **user experience**.



Handle errors gracefully, respond meaningfully, and build **reliable** Mule applications!



5.19

OBJECT STORE



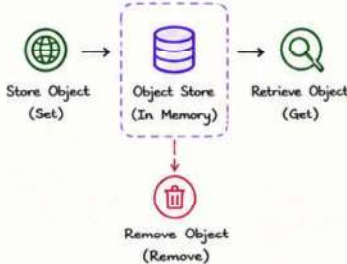
Object Store scope **stores** and **retrieves** Java objects in memory during the application runtime. It enables **caching** and **sharing** data across flows and requests for better **performance** and **efficiency**.

WHAT IS OBJECT STORE?

- ✓ Stores Java objects in memory during runtime.
- ✓ Provides fast access to frequently used data.
- ✓ Enables data sharing across flows and requests.
- ✓ Supports local or distributed storage.
- ✓ Ideal for caching, session management, and reference data.



HOW OBJECT STORE WORKS



Objects are stored as **key-value pairs**. Values can be any Java object.

BASIC SYNTAX

```

<object-store:config name="Object_Store_Config">
  <object-store:object-store
    type="local"
    persistent="true"
    entryTtl="Server Object" />
  </object-store:object-store>
  <object-store:set key="myKey"
    target="Store Object"
    value="#[payload]" />
  <object-store:get key="myKey"
    target="Retrieve Object" />
  <object-store:remove key="myKey"
    target="Remove Object" />
</object-store:config>
  
```

Attribute	Description
key	Unique key used to store or retrieve the object.
value	The Java object to store.
target	Variable to store the retrieved object.

OBJECT STORE IN A FLOW



EXAMPLE FLOW SNIPPET

```

<flow name="user-session-flow">
  <http-listener config-ref="HTTP-Listener-Config" path="/login-data" />
  <set-payload value="#[payload]" />
  <!-- Store object in Object Store -->
  <object-store:set key="userData" value="#[payload]" />
  <!-- Retrieve object from Object Store -->
  <object-store:get key="userData" target="StoredData" defaultValue="Not Object" />
  <logger level="INFO" message="Stored Data: #[vars.storedData]" />
  <!-- Remove object from Object Store -->
  <object-store:remove key="userData" target="Remove Object" />
</flow>
  
```

KEY ATTRIBUTES

Attribute	Description
key	Unique identifier for the object.
value	Java object to be stored.
target	Variable to store the retrieved object.
store-scope	Scope type (local or application (default: local)).
expiration-interval	Interval after which objects expire automatically.
max-entries	Maximum number of objects that is removed.
persistent	Whether the store is persistent (or distributed stores).

BEST PRACTICES

- ✓ Use meaningful keys for easy identification.
- ✓ Avoid storing large objects.
- ✓ Set appropriate expiration and idle times.
- ✓ Remove objects when no longer needed.
- ✓ Use distributed Object Store for clustered environments.
- ✓ Monitor memory usage and clean up regularly.



COMMON USE CASES

- ✓ Caching reference data for faster access.
- ✓ Storing user session or preference data.
- ✓ Sharing data between flows in the same application.
- ✓ Storing temporary results for batch processing.
- ✓ Rate limiting and throttling counters.



CONFIGURATION OPTIONS

Option	Description
store-scope	Store objects in local or distributed scope.
expiration-interval	Time interval to automatically expire objects (ms).
max-entries	Max number of objects allowed in the store.
persistent	Enables persistence across restarts (if supported).
entry-ttl	Default validity period for stored objects.

ERROR HANDLING

Handle errors while storing or retrieving objects.

```

<on-error-continue type="ANY">
  <logger level="ERROR"
    message="Error in Object Store operation:
      #[error.description]" />
</on-error-continue>
  
```



INTERVIEW QUESTIONS

- What is Object Store in MuleSoft?
- How does Object Store differ from variables or properties?
- When should you use Object Store?
- What are the key attributes of Object Store?
- How does distributed Object Store work?



KEY TAKEAWAY

Object Store scope helps you **store** and **retrieve** Java objects in memory during runtime, enabling **caching**, **sharing** data across flows, improving **performance** and **efficiency**.



Use Object Store to **cache**, **share**, and **reuse** data efficiently and build **high-performance** Mule applications!



5.21

CACHE SCOPE



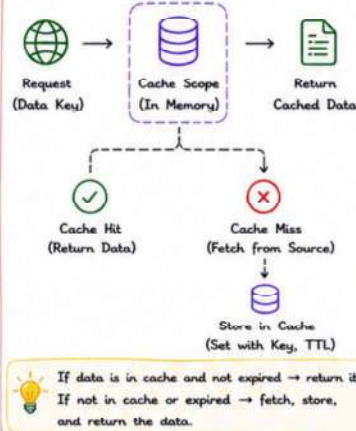
Cache scope **stores** data temporarily in memory for **fast retrieval**.
It **improves performance** by reducing external calls and processing time.
It supports different cache types, TTL expiration, and **size limits**.

WHAT IS CACHE SCOPE?

- Stores data in memory for fast access.
- Reduces external service calls and latency.
- Supports TTL (time to live) expiration.
- Multiple cache types: OBJECT, STRING.
- Configurable size limit and entry count.
- Supports local or distributed caching.



HOW CACHE SCOPE WORKS



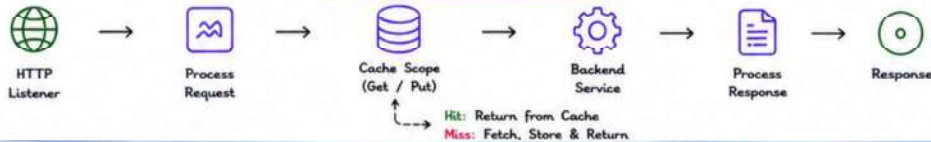
CACHE TYPES

Cache Type	Description
OBJECT	Stores any Java object.
STRING	Stores string values.

BASIC ATTRIBUTES

Attribute	Description
key	Unique key for the cache entry.
value	Value to be stored in cache.
ttl	Time to live for the entry.
timeUnit	Time unit for TTL.
unless	Condition to skip caching.
target	Variable to store the cached value.
defaultValue	Value to return if key not found.

CACHE SCOPE IN A FLOW



CONFIGURATION OPTIONS

Option	Description
entry-count	Maximum number of entries in cache.
time-to-live	Default TTL for entries.
time-unit	Time unit for TTL (SECONDS, MINUTES, etc.).
overflow-policy	Policy when cache is full (e.g., LRU, FIFO).
persistent	Whether cache is persistent (for distributed cache).
store-by-reference	Store object reference instead of value (true/false).



BEST PRACTICES

- Use meaningful and unique cache keys.
- Set appropriate TTL based on data freshness.
- Avoid caching large or frequently changing data.
- Use unless condition to skip null or invalid data.
- Clear cache when underlying data is updated.
- Monitor cache hit/miss ratio for optimization.



COMMON USE CASES

- Caching reference data (countries, currencies, etc.).
- Reducing database or API calls.
- Improving response time for read-heavy flows.
- Storing configuration or lookup data.
- Storing temporary results for batch processing.
- Rate limiting and throttling counters.



ERROR HANDLING

- Use on-error-continue or on-error-propagate.
- Log and handle cache operation failures.
- Set alerts for high error rates.



MONITORING & METRICS

- Track cache hits, misses, and hit ratio.
- Monitor memory usage and entry count.
- Set alerts for high miss ratio or low memory.
- Review and tune TTL and size limits.



INTERVIEW QUESTIONS

- What is Cache scope in MuleSoft?
- How does Cache scope improve performance?
- What is the difference between cache hit and miss?
- How does TTL work in Cache scope?
- How can you clear or remove cache entries?



Cache scope stores data in memory for **fast retrieval**, **reduces latency**, and **improves efficiency**.
Use it wisely with proper TTL, size limits, and key strategy for the best results.



Use Cache scope to **store**, **retrieve**, and **reuse** data efficiently and build **high-performance** Mule applications!



5.22 FILE AND FTP COMPONENTS



File and FTP components enable reading from and writing to files and FTP servers.

They support operations like **read**, **write**, **list**, **move**, **copy**, **delete**, and **streaming** large files efficiently.



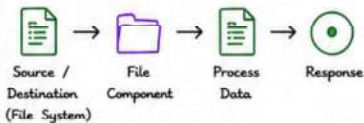
WHAT ARE FILE & FTP COMPONENTS?

- File component works with files in the local file system.
- FTP component works with files on FTP/FTPS servers.
- Support a wide range of operations.
- Handles text, binary, and streaming data.
- Useful for file transfers, data exchange, and batch processing.

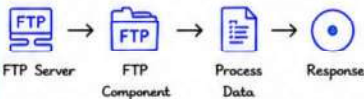


HOW IT WORKS

FILE COMPONENT



FTP COMPONENT



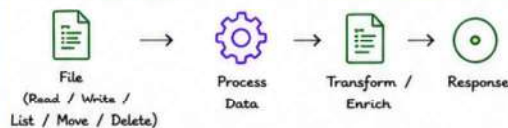
- Read / List** - Get data from File/FTP
- Write** - Create or overwrite data
- Move / Copy / Delete** - Manage files

KEY ATTRIBUTES

- File based (local or FTP).
- Supports operations: **read**, **write**, **list**, **move**, **copy**, **delete**.
- Handles headers, query params, and payloads.
- Supports streaming and timeouts.
- Fully integrated with Mule error handling and security.
- Useful for transfers, exchange, and batch processing.

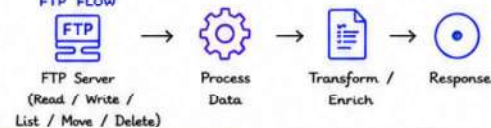


FILE FLOW



FLOW OF FILE & FTP COMPONENTS

FTP FLOW



OPERATIONS SUPPORTED

FILE COMPONENT OPERATIONS

- read** : Reads a file from the local file system.
- write** : Writes data to a file.
- list** : Lists files in a directory.
- move** : Moves a file to another location.
- copy** : Copies a file to another location.
- delete** : Deletes a file.

FTP COMPONENT OPERATIONS

- read** : Reads a file from FTP server.
- write** : Writes data to FTP server.
- list** : Lists files in a directory on FTP server.
- move** : Moves a file on FTP server.
- copy** : Copies a file on FTP server.
- delete** : Deletes a file on FTP server.

CONFIGURATION & EXAMPLE POINTS

FILE CONFIGURATION OPTIONS

- workingDir** : Base directory (file component).
- connection** : Connection working directory.
- recursive** : Enable recursive search.
- maxIdleTime** : Max idle time before removing connection.
- persistent** : Whether the store is persistent.

FTP CONFIGURATION OPTIONS

- host** : FTP server host.
- port** : FTP server port (default 21).
- user** : FTP user name.
- password** : FTP user password.
- passiveMode** : Use passive mode in FTP.
- timeout** : Timeout (FTP) in ms.



BEST PRACTICES

- Use absolute paths for clarity.
- Handle large files using streaming.
- Use passive mode for FTP.
- Close streams properly.
- Handle exceptions and retry if needed.
- Use secure FTP (FTPS) when possible.



COMMON USE CASES

- File based data exchange.
- Loading or unloading flat files.
- SFTP/FTP file transfers.
- Batch file processing.
- Archiving and file management.
- Integration with legacy systems.



KEY TAKEAWAY



File and FTP components make it easy to work with local files and FTP servers. Use them to **read**, **write**, **transfer** files efficiently and handle large data smoothly!



Use File and FTP components to **move**, **manage**, and **process** files efficiently and build **reliable** Mule applications!



5.23

HTTP COMPONENTS



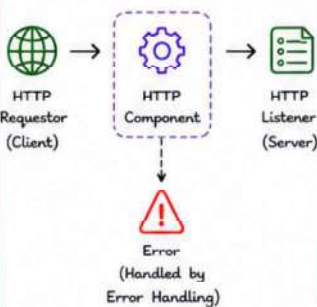
HTTP components allow you to **send** and **receive** HTTP requests and responses. They support **REST**, **SOAP** over HTTP, **streaming**, **authentication**, and **error handling** to build robust and scalable integrations.

WHAT ARE HTTP COMPONENTS?

- ✓ Enable communication over HTTP/HTTPS protocols.
- ✓ Support REST, SOAP, and custom APIs.
- ✓ Handle requests, responses, headers, query params, and payloads.
- ✓ Support authentication, streaming, and timeouts.
- ✓ Built-in error handling and status code support.



HOW HTTP COMPONENTS WORK



HTTP Component acts as a client to send requests and receive responses from HTTP services.

BASIC SYNTAX

HTTP REQUESTOR

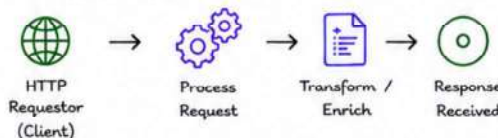
- Sends HTTP requests (e.g., GET, POST).
- Configure path, headers, query params, and timeouts.

HTTP LISTENER

- Listens for incoming HTTP requests.
- Configure path, allowed methods, and timeouts.

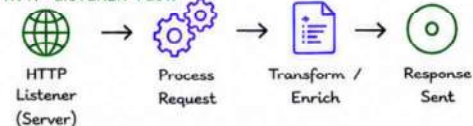
Attribute	Description
config-ref	Reference to HTTP configuration.
method	HTTP method (GET, POST, PUT, DELETE).
path	Resource path.
headers	Request or response headers.
body	Request payload.
responseTimeout	Timeout for responses (in ms).

HTTP REQUEST FLOW



HTTP COMPONENTS IN A FLOW

HTTP LISTENER FLOW



COMMON HTTP METHODS

- **GET** - Retrieve data from the server.
- **POST** - Create a new resource.
- **PUT** - Update an existing resource.
- **PATCH** - Partially update a resource.
- **DELETE** - Delete a resource.
- **HEAD** - Retrieve headers only.
- **OPTIONS** - Get supported HTTP methods.

COMMON STATUS CODES

- 200 - OK - Request successful.
- 201 - Created - Resource created.
- 204 - No Content - Success, no content.
- 400 - Bad Request - Invalid request.
- 401 - Unauthorized - Auth required.
- 403 - Forbidden - Access denied.
- 404 - Not Found - Resource not found.
- 500 - Internal Server Error.

AUTHENTICATION TYPES

- Basic Authentication
- Digest Authentication
- OAuth 1.0 / 2.0
- Client Certificate (mTLS)
- API Key (Header or Query Param)
- Custom Authentication



BEST PRACTICES

- ✓ Use absolute paths for clarity.
- ✓ Set appropriate timeouts.
- ✓ Handle all status codes and errors.
- ✓ Use HTTPS for secure communication.
- ✓ Reuse configurations for better performance.



COMMON USE CASES

- Integrate with external APIs and web services.
- Consume REST and SOAP services.
- Streaming large payloads.
- File transfers and data exchange.
- Batch processing and automation.



ERROR HANDLING

- Use on-error-continue and on-error-propagate.
- Log meaningful error details.
- Return proper HTTP status codes.
- Set alerts for high error rates.



HTTP components simplify communication over HTTP/HTTPS, enabling you to build **scalable**, **secure**, and **efficient** integrations with REST APIs and web services.



Use HTTP components to **connect**, **communicate**, and **create** powerful integrations with the world of APIs!



5.24 WEB SERVICE CONSUMER (SOAP)



Web Service Consumer (SOAP) allows your Mule application to call external **SOAP** web services.

It uses **WSDL** to understand the service contract and supports request/response operations with features like headers, timeouts, security, and error handling.

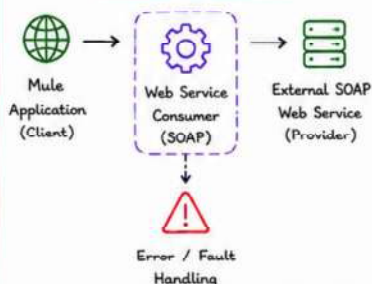


WHAT IS SOAP CONSUMER?

- Invokes operations on external SOAP web services.
- Uses WSDL to generate service metadata.
- Supports request / response communication.
- Handles SOAP headers, fault responses, and attachments.
- Fully integrated with Mule error handling and security.



HOW IT WORKS



The consumer sends a SOAP request based on the WSDL contract and receives a SOAP response or a fault.

KEY FEATURES

- ★ Communicates over HTTP/HTTPS.
- ★ Supports REST, SOAP, and custom APIs.
- ★ Handles headers, query params, and payloads.
- ★ Supports authentication, streaming, and timeouts.
- ★ Built-in error handling and status code support.



FLOW OF SOAP CONSUMER



SUPPORTED OPERATIONS

- ✓ read : Reads a file from the server.
- ✓ write : Writes data to a file.
- ✓ list : Lists files in a directory.
- ✓ move : Moves a file to another location.
- ✓ copy : Copies a file to another location.
- ✓ delete : Deletes a file.

COMMON HTTP METHODS

- GET : Retrieve data.
- POST : Create a resource.
- PUT : Update a resource.
- PATCH : Partially update a resource.
- DELETE : Delete a resource.
- HEAD : Retrieve headers only.
- OPTIONS : Get supported methods.

COMMON STATUS CODES

- 200 : OK
- 201 : Created
- 400 : Bad Request
- 401 : Unauthorized
- 403 : Forbidden
- 404 : Not Found
- 500 : Internal Server Error

SECURITY SUPPORT

- ✓ WS-Security (UsernameToken).
- ✓ Basic authentication over HTTPS.
- ✓ Client certificates (mTLS).
- ✓ API Key (Header or Query Param).
- ✓ Custom SOAP headers for tokens.



BEST PRACTICES

- ✓ Use WSDL from a reliable source.
- ✓ Set appropriate timeouts.
- ✓ Handle all status codes and errors.
- ✓ Use secure endpoints (HTTPS).
- ✓ Reuse configurations.



COMMON USE CASES

- ✓ Integrate with external enterprise systems.
- ✓ Consume legacy SOAP services.
- ✓ Communication with CRM, ERP, or mainframe systems.
- ✓ Batch processing and data exchange.



Web Service Consumer (SOAP) enables your Mule application to reliably call external **SOAP** services using **WSDL**, handle responses, faults, and security efficiently.





Thank You!

For reading and learning with us.

We hope this **MuleSoft Anypoint Platform Components Guide – Volume 1** has helped you build a stronger foundation and boost your integration skills.

Keep practicing, keep building, and keep innovating.
You're now one step closer to becoming a **MuleSoft integration expert!**



 **Thank you for being a part of this journey!**
Your curiosity and commitment drive your growth and inspire continuous learning.

 **Volume 2 is coming soon!**

More components, more insights, and more real-world scenarios to level up your integration expertise.

Stay tuned!